Technology
Arts Sciences
TH Köln

Jamestown

Password- / Keyless authentication

Bachelor Thesis

Computer Science & Engineering

TH Köln

Mert Torun

Prof. Dr. Michael Silverberg *(First reviewer, TH Köln)*

Frank Mördel *(Second reviewer, Jamestown US-Immobilien GmbH, Cologne)*

June 2021

Technology
Arts Sciences
TH Köln

Jamestown

Bachelor Thesis

Title: Password- / Keyless authentication

Reviewers:

- Prof. Dr. Michael Silverberg *(First reviewer, TH Köln)*
- Frank Mördel *(Second reviewer, Jamestown US-Immobilien GmbH, Cologne)*

Abstract: The aim of this academic thesis is to investigate and identify the usage possibilities, security and convenience of **Password- / Keyless authentication**. This is performed within the Web presence of Jamestown US-Immobilien GmbH to specify its use. These requirements will be analyzed in the context of authenticating transactions in the Jamestown Online (JTO, **[A1]**) application, presenting a passwordless approach.
In the first part, an introduction to Jamestown US-Immobilien GmbH, Fast IDentity Online (FIDO, **[A2]**) Alliance and passwords is given. These will show off their company profile, business field and the increasing demand of a convenient authentication in applications like JTO. Also showing off the limitations of password-based authentication, comparing them to the pros and cons of similar approaches.
The thesis subsequently identifies various methods in detail, and further outlines the structure of a passwordless-based approach in contrast to a password-based approach.
In conclusion, the thesis argues that convenient access to applications is essential for a company to stay competitive and presents an example for JTO.

Keywords:

```
{
        Secure;
        Convenient;
        Passwordless;
        Distributed;
        Authentication;

};
```

Restriction notice: The access to this thesis is restricted until 30th June 2024.

Date: 30th June 2021

Table of Contents

1 Introduction **5**

1.1 FIDO Alliance 6

1.2 Jamestown 7

2 Passwords **10**

2.1 Knowledge-based 12

2.2 Token-based 13

2.3 Biometric-based 15

3 Cryptography **17**

3.1 Fundamental Model 21

3.2 Cryptographic Algorithms 22

3.2.1 Private-Key Cryptography 22

3.2.2 Public-Key Cryptography 25

3.2.3 Digital Signatures 29

3.2.4 Steganography 32

3.3 Key Distribution 33

3.3.1 Diffie-Hellman Key Exchange 34

3.3.2 Authentication of Public Keys 35

3.4 Transactions 37

4 FIDO Alliance Specifications **38**

4.1 Universal Second Factor 39

4.2 Universal Authentication Framework 40

4.3 Client to Authenticator Protocols 41

5 From inconvenient to convenient **42**

5.1 Jamestown Online 49

5.2 Online-Zeichnungsschein 51

6 Conclusion **52**

7 Appendix **53**

8 List of Abbreviations **63**

9 List of Figures 65

10 List of Tables 67

11 References 68

12 Statutory Declaration 70

13 Blocking Notice 71

1 Introduction

In this paper, I am presenting to you the modern authentication standards, which are representing the current state-of-the-art convenient and secure way of digital communication. Exchanging a generic message **[m]** with a generic system **[x]** by a generic actor **[y]** using method **[z]** and key **[k]** is done utilizing these techniques. The examples I will be presenting to you are partly based on the work of Fast IDentity Online Alliance — abbreviated name FIDO Alliance. FIDO is an open industry association who is developing and promoting authentication standards that bear us toward passwordless, convenient and secure authentication, enabling them as standards.



Figure 1, **[F1]**: FIDO Alliance and Jamestown corporate identity collage

To start off with, I will present to you an overview of the involved businesses in the first chapter. These are FIDO Alliance and Jamestown as companies and some information to gather a basic clue of their work fields. Following this up with the fundamental knowledge of passwords divided into physical-based and virtual-based which is required for authentication later on.

Furthermore, Cryptography will be the substantial part of this thesis, supplying the fundamental knowledge of the algorithms and the model used. How keys are exchanged, messages and transactions signed will be a core part of this thesis covered in chapter three. These are required to establish a responsible, modern and convenient authentication method, as passwordless-based authentication is presenting itself.

As this thesis is partly based on the work of the FIDO Alliance, I will get into their specifications in the fourth chapter. Their open authentication standards will be presented in detail, and their pros and cons will be compared to each other in a structured and logical overview.

Conclusively, the thesis will transition to an implementation approach of passwordless authentication at Jamestown Online and show off a generalized cryptographic and specialized approach for convenient authentication standards. The prime reason for this is Jamestown wants to offer a convenient authentication for its customers, therefore offering them more variety than the competition.

1.1 FIDO Alliance

FIDO Alliance is an open industry association. An industry association typically represents a non-profit organization. These typically provide its active members with an extensive range of active and passive services, including support and training, as well as lobbying on their behalf. In the figure underneath, you can receive a glimpse of the FIDO Alliance members, which surely is impressive as big company names come in handy.



Figure 2, **[F2]**: FIDO Alliance members overview cutout

It's significant to retain non-profit associations like FIDO Alliance. For sharing knowledge about specific industry branches and typically provide their members with useful resource pools, establish global conferences and meet-ups — for example **Authenticate 2020 Conference** (ref. **[14]**) hosted in November 2020 covering policy discussions about the general data protection rule as well as standards and technical implementations for identity verification.

This thesis will cover the following FIDO Alliance specifications:

Universal Second Factor

Universal Authentication Framework

Client to Authenticator Protocols

Which are representing the modern and openly accessible state-of-the-art possibilities for authentication based problem resolutions — offering everyone the same chances in terms of security enhancements by multiple-eye principle.

1.2 Jamestown



Figure 3, **[F3]**: Ponce City Market, Atlanta

Since the company was founded in 1983, real-estate investments with a key focus on the United States have determined the business. Jamestown consists of two main sister companies in Germany and the USA: Jamestown US-Immobilien GmbH in Cologne and Jamestown L.P. in Atlanta. Shareholders of these companies are Christoph and Ute Kahl as well as Michael Phillips and Matt Bronfman. In 2020, the company managed real estate assets worth 12 billion US dollars for German private and institutional investors from all over the world.



Figure 4, **[F4]**: Office space in Ponce City Market, Atlanta



Figure 5, **[F5]**: Jamestown 31 Levi's Plaza, San Francisco

The Jamestown Group consists of **three affiliated entities** in Europe and in the USA, which are separate legally and in terms of function:

Jamestown US-Immobilien GmbH is responsible for investor communications and support services to investors in Germany. Since mid-2014, Jamestown US-Immobilien GmbH has been an external capital management company (Kapitalverwaltungsgesellschaft, (KVG, **[A3]**)) pursuant to KAGB supervised by the Federal Financial Supervisory Authority (BaFin, **[A4]**). The KVG undertakes the management of Jamestown 29, Jamestown 30, Jamestown 31 and if appropriate, other closed-end limited investment partnerships, especially fund management and risk management. The wholly owned subsidiary Jamestown Vertriebs-GmbH has been appointed to sell the units and coordinate sales. Around 50 employees in total, who have acquired many years of experience in the management of US real estate funds, work at the registered office in Cologne.

Jamestown, L.P. with a registered office in Atlanta, USA, manages the business of all US real-estate funds set up by the Jamestown Group prior to 2014. Jamestown L.P. is properly registered as an Investment Adviser in the USA, where it is supervised by the Securities and Exchange Commission (SEC, **[A5]**). Around 160 staff are employed at the Jamestown headquarters in Atlanta and in the local offices in New York, Boston, Los Angeles and San Francisco. Jamestown, L.P. acts as Asset Manager for Jamestown 29, Jamestown 30 and Jamestown 31.

Jamestown Europe with its headquarters in Cologne was launched in 2018 with the goal of acquiring and managing large scale, historic industrial, adaptive reuse projects across Europe.

"We develop competitive locations for tomorrow by making properties and their surroundings attractive places for tenants and visitors. In this way we achieve a sustainable increase in value from which our investors benefit."

- C. Kahl, Shareholder and founder of the Jamestown US-Immobilien GmbH



Figure 6, **[F6]**: Jamestown 31 Garco Mill, South Carolina

A word about *Jamestown and economic sustainability*: Jamestown undoubtedly possesses extensive expertise in the energy-efficient and environment-friendly construction and operation of historic properties. Jamestown Green carefully analyzes and implements energy-saving measures in portfolio properties and endeavors to pursue an environmentally sound policy. The certification of sustainability also increases the prestige of the property among tenants and potential buyers.

2 Passwords

Secure passwords and modern cryptography both represent essential parts in systems as **control mechanisms** against malicious intruders to prevent cracking in a variety of possible ways. Therefore, we typically need password-like unique identifiable tokens to sufficiently establish confident access to a system and properly utilize cryptographic methods. This type of confidential access has its roots early in history. As a prime example, when opening doors, you are using a unique key or a magic word to access something and verify your claimed identity. This elementary principle of unlocking a door or remembering a passphrase can be translated from physical access keys, to digital access keys. This kind of digital access key and its abbreviations will be covered in the subsequent chapters.

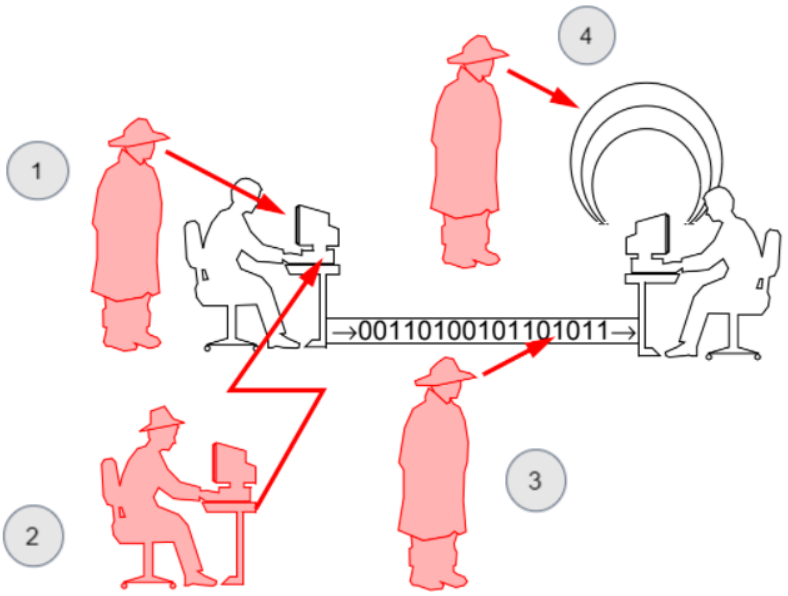


Figure 7, **[F7]**: Possible Cracker Attacks

In the figure above the different possible cracker attacks which can either be executed by weak points in internal (**1**, **2**) or external (**3**, **4**) security systems are displayed. Internal security obtains a protection against unauthorized actions of a potential user or process within the system itself, enforced inside the system and is called **access control**. External security is defined as **admission control** to a system, which proactively protects against potential intruders from the outside and is enforced at the external border of the system.

In modern day-to-day security systems, it is critical to allow admission and access control both working together hand in hand smoothly in a network operated system. A system will merely offer a high security standard if there are control mechanisms for the external as well as the internal system. For example, it doesn't make much sense to secure a web server with a password but leave it in a public place where everyone can carry it away.

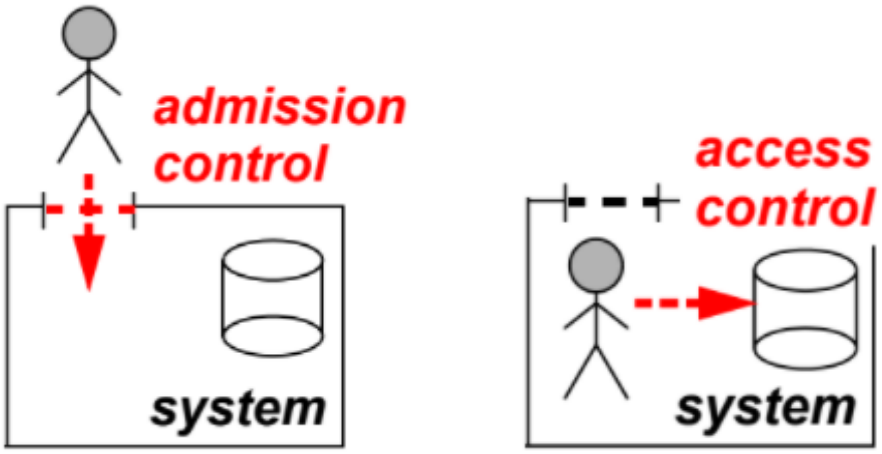


Figure 8, **[F8]**: Admission versus access control in local security systems

Admission control is forcing every user to pass a specific access path or access routine to authenticate himself and verify his unique identity. Specific access paths, for example, can in common be edificial structures like a locked computer room with a Faraday cage or safes containing sensitive storage media. Firewalls or a total system isolation from the modern internet can represent specific access paths, which properly protects against possible attacks from the network. This chapter is focusing on login procedures itself which, in theory, should be unavoidable to access the system and authenticate the user. The login procedures can either represent knowledge or biometric-based authentication, which will be adequately explained in the sub-chapters later on.

Access control naturally assumes that admission control already happened beforehand and that there are only authorized and authenticated users within the system. These users may legitimately claim various rights and possess various active roles executing these rights. This means independent rights provide privileged access to examine specific data and execute specific operations.

2.1 Knowledge-based

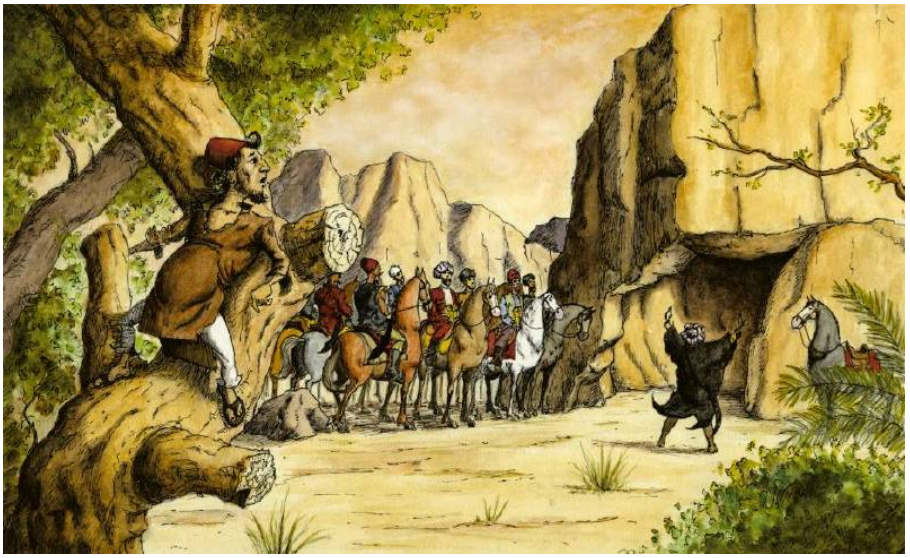


Figure 9, **[F9]**: Ali Baba and the Forty Thieves fairy tale colorized

Knowledge-based techniques presumably are one of the most established known techniques for proper verification of specific individuals, modern machines or specific groups between each other in any possible combination. Originating from **Ali Baba and the Forty Thieves fairy tale** (ref. **[15]**) based on real occurrences, the first recorded password in history is falsely known as open sesame. In fact, **Sésame, ouvre-toi** (French, original) / **Sesame, open yourself** (English, transcribed) correctly was the original terminology.

Generally speaking, knowledge-based techniques are typically using unique passwords in the form of arbitrary strings of characters including letters, digits and other symbols. As well as personal identification numbers (PIN, **[A6]**) which are typically constrained to be numeric but not only, however. Therefore, excluding unique biometric traits and hardware token while including anything you could theoretically recognize to verify your claimed identity is the specific intent of knowledge-based techniques. These passwords and pins are also sometimes also called **magic words** in academic literature. This is because passwords don't have to represent senseful words themselves. Indeed, a non-word password in the dictionary sense may be harder to guess correctly, which represents a desirable property of secure passwords.

These magic words are vulnerable to possible abuse once leaked. However, they can easily be changed by simply informing all associated parties from a credible source about the change. Therefore, rendering these techniques as a viable option for multi factor authentication (MFA, **[A7]**) based technologies. It can be efficiently utilized as a strong first authentication and backup authentication way. For the prime example, having an additional security layer to give users convenient access to a multi factor system after failing to use biometrics.

2.2 Token-based

In token-based verification, a physical existing real-world device replaces the place of a non-physical existing virtual-world password. These portable devices typically possess specific security certificates, to ensure they sufficiently meet the required security specifications. Naturally speaking, they act like an electronic door key to access something. Therefore, a user is required to possess this specific physical item as correctness proof for his claimed identity, which he registered at the enrollment process previously. The physical item is typically required at least once at login and occasionally, depending on the single sign on (SSO, **[A8]**) method implemented. This specific item can in common be anything unique like a physical key, chip card, memory stick and many more items you could think of.



Figure 10, **[F10]**: Google Titan hardware security keys product line-up 2021

Token-based verification undoubtedly possesses a high security versus account takeovers from phishing attacks. Even if someone stole your secure password, they won't be capable of logging into your compromised account as it is secured by two factor authentication (2FA, **[A9]**). This, in common, instantly adds an extra security layer by ensuring this 2FA. As the token owner, you inseparably connect the hard token to the system and your secure password, which you want to enter to gain access to its service. However, without the specific token that was registered at the enrollment process as a second distinct security layer, the access is blocked. Since the specific token inevitably — at any login attempt — generates a new unique cryptographic proof of identity for the owner.

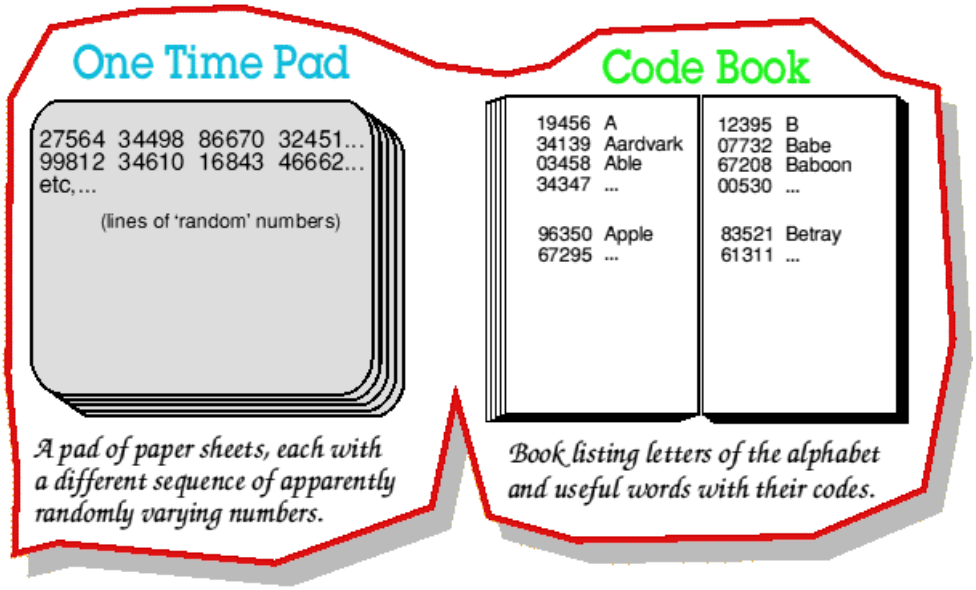


Figure 11, **[F11]**: One-time pad

In direct contrast to knowledge-based and token-based methods, a One-time pad (OTP, **[A10]**) represents a possible method scientifically proven to be information-theoretically secure. This means a system is even secure in exceptional cases where a potential adversary has unlimited computing power and time. Therefore, rendering the system crypto analytically unbreakable, if the adversary doesn't obtain sufficient information to break the key encryption. However, only properly used OTPs, which naturally provide no reliable information about the original message to a cryptanalyst (except the maximum possible length — since the unique key can be longer than the message itself by using padding but has to be at least as long as the whole message including possible paddings) are secure. Only if all four conditions are fully satisfied, the OTP is unbreakable and those are as following:

The key must be independent of the message and must maintain a uniform distribution

The key must be at least as long as the message

The key must never be reused in part or whole

The key must be kept secret by the involved parties

It is also proven, that any cipher with the desirable property of perfect secrecy must also use keys with the same requirements as stated above for OTPs.

However, One-time pads are impractical for most real-world applications because of their highly complex key management and low scalability. Instead, we will focus on more convenient cryptographic but less secure methods in the upcoming chapters.

2.3 Biometric-based

Through unique biological traits on the user's body themselves possessed as a specific property, verification can also happen. Speaking about **iris**, **voiceprint**, **fingerprint**, **facial characteristics** and many more unique characteristics to be compared against the stored biometric data in biometric authentication systems. This modern type of complex password, or also called key, is unique and offers high security paired with a high convenience user-end wise. However, this modern technique typically remains extremely vulnerable if stolen once, since you can't change your fingerprint or palm veins easily as a knowledge-based password. Biometric authentication is sometimes also called **realistic authentication** (ref. **[6]**, **[7]**) in computer science literature.

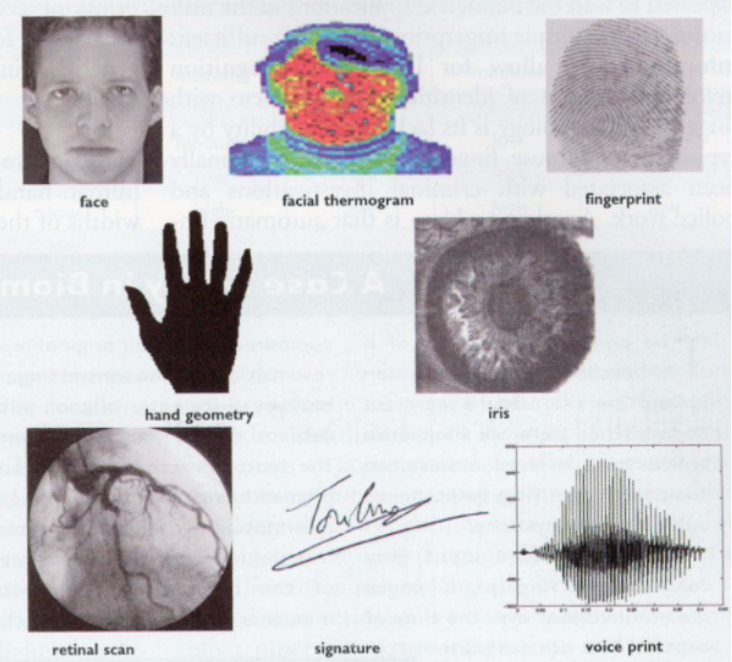


Figure 12, **[F12]**: Biometric Authentication

This modern type of complex and secure password is immensely application dependent. We will distinguish that certain biometrics will be better than others based on the required levels of practical convenience and security by the desired application. This means precisely no particular biometrics conforms to all the unique requirements of every possible application.

To properly weigh the specific factors to properly select particular biometrics for use in specific applications, there are currently seven contributing factors to be oriented on (ref. **[16]**). These are taken into consideration, when determining the best fit biometrics for specific applications:

Universality — *everyone operating the system should possess the trait*

Uniqueness — *the trait should be distinguishable from other persons*

Permanence — *relate to trait variety over time*

Measurability — *trait measurement rate in terms of collectability*

Performance — *accuracy, speed and robustness of technology used*

Acceptability — *persons willing to have their biometric traits captured and assessed*

Circumvention — *ease with which a trait might be imitated providing a substitute*

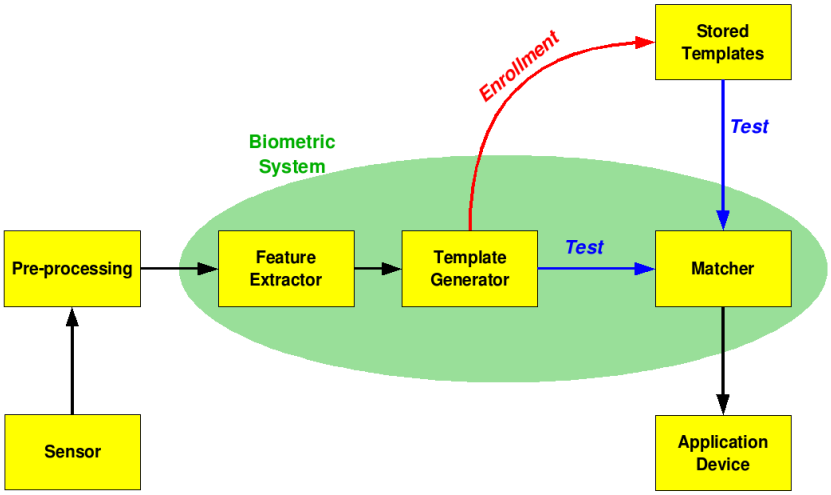


Figure 13, **[F13]**: Main logical block of a biometric system simplified

The figure above illustrates the two fundamental modes of a generic biometric system. Foremost, it is critical that the biological trait used to be processed represents a high collectability. Therefore, meaning that in addition to the ease of successful acquisition or accurate measurement of the unique trait, the necessary data should be in a specific form that permits subsequent processing and extraction of relevant future sets.

An application device traditionally represents your mobile device to offer you convenience in authentication, which can in common be anything from a modern smartphone to a portable laptop. Such devices usually contain an inbuilt sensor, which typically represents an image acquisition system. These systems can capture various biometrics at once and are already widespread, offering an inexpensive, convenient solution for application device manufacturers and service providers to enable biometric authentication techniques.

3 Cryptography

Modern cryptography in common is the encryption of data. Therefore, information is secure even with the unauthorized access to storage media or eavesdropping into a data transmission. Further usage of cryptography typically includes the mutual authentication of persons, services and signing of messages. All of these use-case scenarios will be covered in the subsequent chapters of this thesis, to enable a convenient and secure passwordless authentication later on. In academic literature as well as in this thesis the names Alice (A, [A11]) for the sender, Bob (B, [A12]) for the intended recipient and Eavesdropper (Eve, [A13]) for the adversary are being used. This naming convention is traditionally used to describe simplified use-case scenarios.

$$\text{Cryptology} = \text{cryptography} + \text{cryptanalysis} \tag{3.1}$$

Whereas, cryptanalysis covers the cracking of the encryption of messages and making the plaintext readable back from ciphertext, we are intentionally trying to make this decryption process complexity as hard as possible. Using cryptography we are progressively moving the point-even for crackers to such an elevated level that cracking becomes unrentable anymore in means of time consumption. Therefore, modern cryptography in common is — as of today — based on mathematical problems like the integer factorization or the discrete logarithm problems, so there are deep connections with abstract mathematics. Theoretically it's possible to break into any well-designed system, however as already mentioned beforehand we are trying to make this as infeasible as possible for any potential adversaries. Combining both, cryptography and cryptanalysis is succinctly summarized under the key term of the cryptology. However, in this thesis, we will be focusing on modern cryptography and their various methods for encrypting messages and exchanging keys — which are both crucial techniques for properly implementing a passwordless approach later on.

In the subsequent chapters, we are going to explore various pragmatic approaches for modern cryptography. Where I will be starting off with the fundamental model, which is no more than explaining how encryption works in its most essential way. Therefore, this model is unbound to any specific algorithm or method — just a theoretical show-off. Initially, we have to make sure everyone comprehends the following:

Private-Key Cryptography = **symmetric encryption** (3.2)

Public-Key Cryptography = **asymmetric encryption** (3.3)

Essentially, these two approaches are fundamentally different in the practical way encryption keys are created and how key exchanging processes work — but both methods are useful in their own way with pros and cons in various scenarios, we will get into detail with this in the following chapters. Let's start off with:

Cryptographic **hash** functions

Cryptographic hash functions represent an essential tool of modern cryptography. Password verification commonly relies on the frequent use of cryptographic hashes to securely store and verify passwords later on. Storing all user passwords in clear text can result in a massive security breach if the password file would be compromised and should under no exceptional circumstances be done. This security risk can be easily reduced by only storing the hash digest of each password, rendering any compromised password file useless for potential attackers. Keep in mind that a password reset function is mandatory when using password hashing, as retrieving the original password is impossible. The following represents a list of possible use-case scenarios for cryptographic hash functions:

Verifying the integrity of messages and files

Signature generation and verification

Password verification

Proof-of-work

File or data identifier

In this thesis, we will be focusing on the first three named use-cases from the list above. A cryptographic hash function is precisely a mathematical algorithm which maps data of arbitrary size to a bit array of a determined size — this means, the algorithm consistently produces the same output length given multiple inputs with different length parameters.

Such hash functions are defined as:

One-way functions

One-way functions represent a type of function easy to efficiently compute on every given input, but hard to invert. It is to be mentioned that the absolute existence of such one-way functions however lacks viable proof. This problem, however, is bound to other scientifically based problems. Direct superior to this, the P versus NP problem is assuredly a complex topic of modern computer science. Which undoubtedly has to be resolved beforehand to scientifically prove or disprove one-way functions.

Getting back to hash functions, even if a cracker would attempt to try and execute a dictionary attack on our hash values with or without a rainbow table, it would be easily countered by adding a custom salt value to the used public known hash function. This specific digest value instead of a cleartext password would render these coordinated attacks useless. A dictionary attack represents an attempt by a cracker to try and login using a specific set of given plaintext passwords like qwertz, qwertz1, qwertz2 and so on. However, a rainbow table contains plaintext passwords already with encryption. Ordinarily, these attacks are not working as there in common is a specific counter on the login attempts in web services. This counter blocks the access after three failed attempts, for example — this won't be part of the thesis but should be mentioned, however.

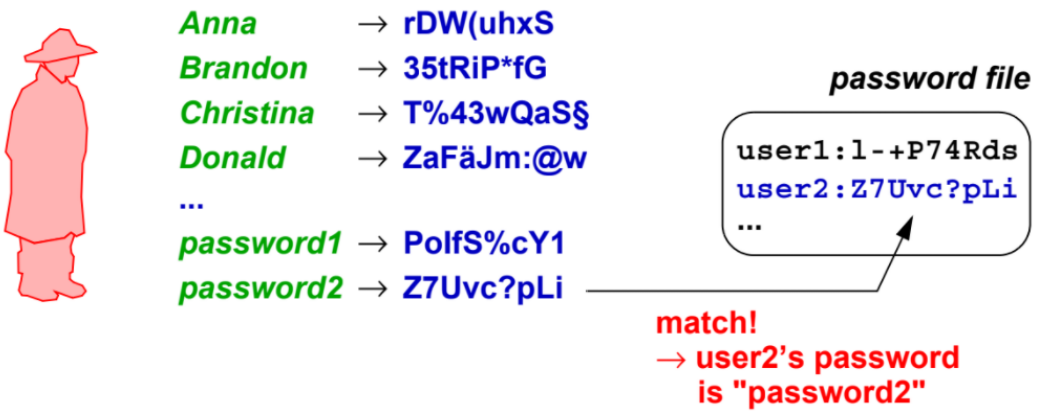


Figure 14, **[F14]**: Dictionary attack simplified example

After getting to identify and utilize the various tools we have to encrypt and decrypt messages, deliver and receive keys in a convenient, secure and scalable way, the thesis will transition to signing messages and digital signatures — also called the **identity verification** — which is solved by using cryptographic hash functions, they are type of cryptographic algorithms.

How do you make sure, the sender of a message is who he possesses to be?

This is achieved by making use of the various algorithms to sign a message using a digital signature, which only the sender is able to sign. This will be covered in one of the latter subchapters, using public-key encryption to make use of to sign messages. A unique signature authenticates both the sender and the message. It also means the message wasn't modified during the transmission over possibly insecure channels.

Before we start off with cryptography, let me get back to passwords for a moment. In the table below, I want to demonstrate the advantages and disadvantages of each method we got to know in the previous chapter. In practice, it's best to use a combination of the techniques in order to achieve a convenient and secure access — MFA.

	Advantages	Disadvantages
Knowledge-based	Simple to use and implement	Only as secure as the user deploying it
	Easy to change once leaked	Can be lost, stolen or even forgot by the user
Token-based	Highest possible sole-authentication technique security offered as new correctness proof is generated every time at login attempt	Can be lost, stolen, outdated or become defective over time
Biometric-based	Simplifies authentication as there are no passwords	Can't be changed
	High convenience to use	
	Every user possesses unique traits	

Table 1, **[T1]**: Passwords overview advantages and disadvantages of various techniques

3.1 Fundamental Model

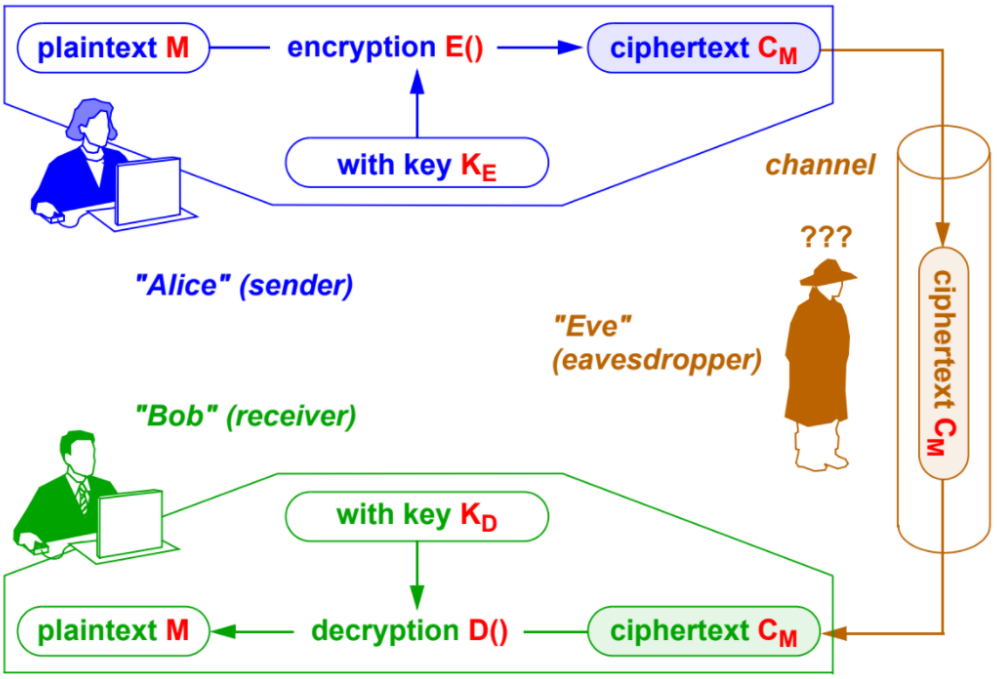


Figure 15, **[F15]**: Fundamental Model visualized

The figure above shows how encryption works in a generalized way without defining any specific encryption function for now. Encryption and decryption functions use **keys** as their critical parameters:

$$\text{Encryption is done with } \mathbf{K_E} \text{ using } C_M = E(M, \mathbf{K_E}) \qquad (3.1.1)$$

$$\text{Decryption is done with } \mathbf{K_D} \text{ using } M = D(C_M, \mathbf{K_D}) \qquad (3.1.2)$$

Note: The key values are crucial for secure encryption and decryption. C_M cannot be decrypted if K_E and K_D are both kept secret.

The functions E() and D() can and should be published. This means the security of a cipher system must depend solely on the concealment of the key, but not on the concealment of the algorithm itself (ref. **[22]**). In the subsequent chapters, we will see that sometimes even K_E can be published — this is termed public key cryptography.

3.2 Cryptographic Algorithms

3.2.1 Private-Key Cryptography

Private-Key Cryptography is one of the oldest known cryptographic algorithms. This type of algorithm is also known as:

Secret-Key Cryptography

Symmetric Cryptography

Symmetric Encryption

This is a symmetric algorithm type, because:

$$K_E = K_D \tag{3.2.1.1}$$

Which means that the same key is used for encryption and decryption. Sometimes they can be different from each other, but these keys do have the property that K_E can be easily calculated from K_D and vice versa. One of the main drawbacks of this method is that the involved parties require access to the secret key, in comparison to public-key cryptography. There are two subtypes of this algorithm workflow:

Stream ciphers

Block ciphers

This thesis will only cover block ciphers, as these, in comparison to stream ciphers, take a number of bits and encrypt them as a single unit — like the Advanced Encryption Standard (AES, **[A14]**) introduced in 2001 does. Main advantages of the AES are the following properties:

Security

Public design process

Speed

Simple implementation

As already mentioned before: The security of a cipher system must depend only on the concealment of the key but not on the concealment of the algorithm — which is true for AES since it utilizes the public design process, which makes its source codes open for everyone. This in conclusion means that anyone can contribute to the AES algorithm and identify weaknesses to fix.

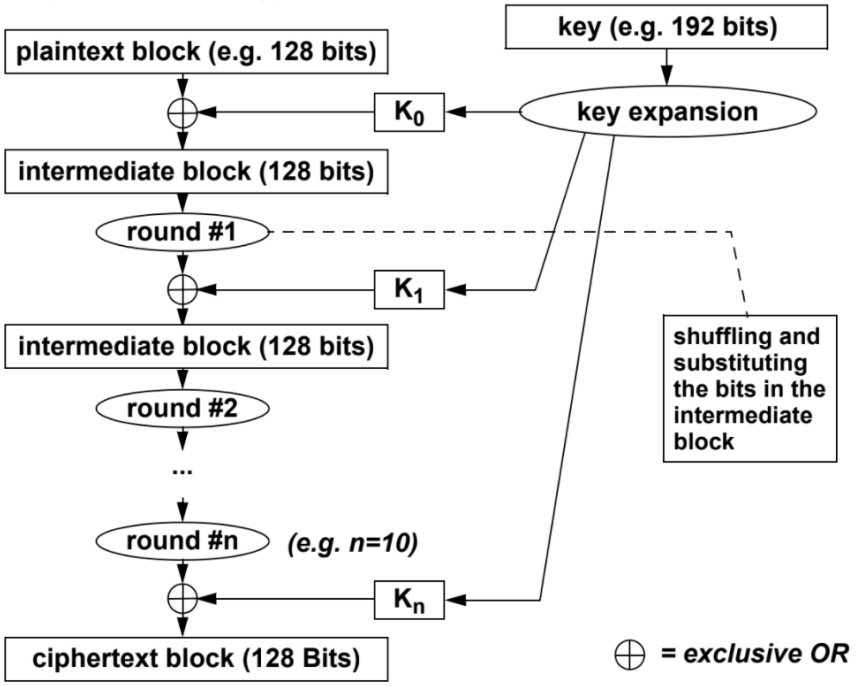


Figure 16, **[F16]**: Steps of AES Encryption

In the figure above, you can see the steps of AES Encryption using a 192 bits long key, keys in AES typically have either 128, 192 or 256 as key size. AES is based on the design principle known as a substitution-permutation network and is efficient in both soft- and hardware. A substitution-permutation network is a series of linked mathematical operations which are being used by AES and other block cipher algorithms. Since AES is a variant of Rijndael, the specification with the block and key size are per se any multiple or 32 bits, with a minimum of 128 and a maximum of 256 bits. The key size used for an AES cipher specifies the number of transformation rounds that convert the input into the ciphertext. The number of rounds is as following for the keys:

10 rounds for 128-bit keys

12 rounds for 192-bit keys

14 rounds for 256-bit keys

But why? Let's take a closer look at how AES operates. AES operates on a 4 x 4 column-major order array of bytes, termed the state. This means, that the following is given for AES using 192-bit keys:

4 x 4 = 16 for our state (3.2.1.2)

192-bit divided by state = 12 rounds (3.2.1.3)

AES: High-level description of the algorithm (ref. [23])

1. KeyExpansion

The round keys are derived from the cipher key using the AES key schedule. Which is expanding a short key into a number of separate round keys.

2. Initial round key addition

I. AddRoundKey — each byte of the state is being combined with a byte of the given round key using bitwise XOR operations.

3. 9, 11 or 13 rounds:
    a. SubBytes — non-linear substitution step where each byte is replaced with another according to a lookup table
    b. ShiftRows — a transposition step where the last three rows of the state are shifted cyclically a certain number of steps
    c. MixColumns — linear mix operation which does operate on the columns of the state, combining the four bytes in each column
    d. AddRoundKey
4. Final round (making 10, 12 or 14 rounds in total):
    a. SubBytes
    b. ShiftRows
    c. AddRoundKey

See the complete definition of AES in ISO/IEC 18033-3: Block ciphers for further low-level explanation of the algorithm.

3.2.2 Public-Key Cryptography

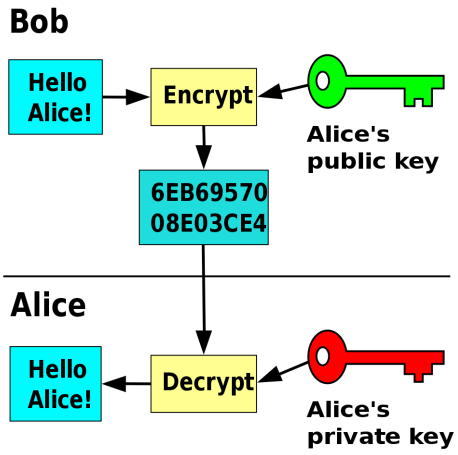


Figure 17, **[F17]**: Public-Key Cryptography whereas Alice represents the receiver and Bob the sender

Public-Key was designed after Private-Key in order to fix the main design flaw, which is the complex key-exchanging of keys in Private-Key cryptography when scaling the systems or messages up. By contrast, in a public key system, the public keys can be disseminated widely and openly, and only the corresponding private keys have to be kept secret by its owner. This is also termed as:

Asymmetric Cryptography

Asymmetric Encryption

This is an asymmetric type, because:

$$K_E \neq K_D \qquad (3.2.2.1)$$

Which means that different keys are used for encryption and decryption. While K_E is public, to be used by the sender and K_D is private, to be used by the legitimate receiver only. The decryption key K_D cannot be calculated from K_E in practice within a limited time. Therefore, in this type of system, anyone can encrypt a message using the receiver's public key, but this message can only be decrypted with the corresponding private key of the receiver, which is kept secret.

In practice, public-key cryptography is used to exchange private-keys using the public-key infrastructure. This does, at the same time, allow for complex key management from public-keys but use the benefits of easily computable encryption and decryption of private-keys.

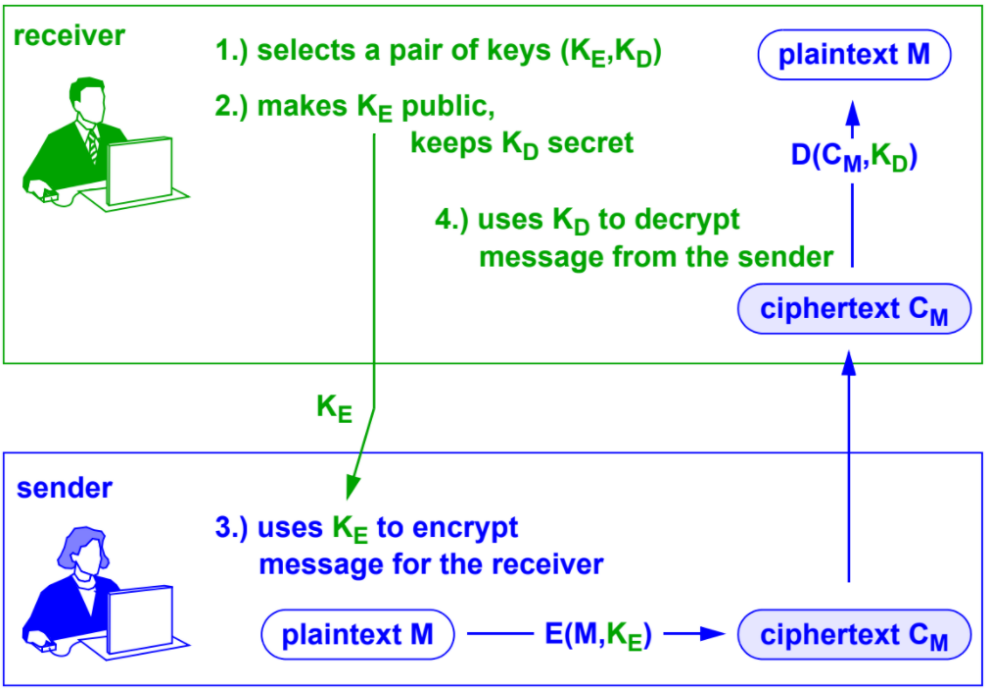


Figure 18, **[F18]**: Public-Key — The General Model

K_E and K_D belong together, which means that messages encrypted using K_E can only be decrypted using the corresponding key K_D. Only the receiver should be in possession of the key K_D using it to encrypt messages which were encrypted using K_E. The main advantage of this structure is that K_E can freely be published on a Web page or in an Email, for example — without having security flaws. However, with public-key comes a higher complexity than with private-key in terms of implementation and a considerably longer execution time. As already mentioned, in practice public-key cryptography is used only for the secure transmission of a key K_{priv} and private-key cryptography is used for the secure transmission of the message itself, which combines the best of both worlds.

The question coming up for this is, how do we validate the authenticity of K_E? Was K_E indeed published by the authorized receiver, or was it published by an Eve? The solution to this is the certification of keys using a certification authority, see the later sub-chapters for an answer to this.

RSA Algorithm: Definition

RSA is a public-key encryption algorithm, invented by Rivest, Shamir, Adleman in 1978. The algorithm itself is based on the Euler's φ-function:

$$\varphi(n) := \left|\{a \in \mathbb{N} \mid 1 \leq a \leq n \wedge \mathrm{ggT}(a,n) = 1\}\right| \qquad (3.2.2.2)$$

The number of all numbers smaller than or equal to n that have no common divisor with n. The properties used by RSA are as following:

$$\varphi(p) = p-1 \text{ for a prime number } p \qquad (3.2.2.3)$$

$$\varphi(a \cdot b) = \varphi(a) \cdot \varphi(b) \text{ if } \gcd(a,b) = 1 \qquad (3.2.2.4)$$

Note: GCD = Greatest Common Divisor.

In RSA, the receiver defines both keys:

K_E for the message encryption

K_D for the message decryption

For this, the receiver does select two large prime numbers p_1 and p_2. Then he calculates:

$$n = p_1 \cdot p_2 \qquad (3.2.2.5)$$

Select the two keys d and e, such that:

$$d \cdot e \bmod \varphi(n) = 1 \qquad (3.2.2.6)$$

First select:

$$d \text{ with } \gcd(d, \varphi(n)) = \gcd(d, (p_1-1) \cdot (p_2-1)) = 1 \qquad (3.2.2.7)$$

Then select:

$$e \text{ with } \gcd(d \cdot e, \varphi(n)) = 1 \qquad (3.2.2.8)$$

Then the receiver makes e and n public but keeps d secret.

Encryption / decryption scheme of RSA

In order to encrypt a plaintext, the sender codes the plaintext as an integer number: m. E.g. all the ASCII numbers of the text's character. Calculate ciphertext as following:

$$y = m^e \bmod n \qquad (3.2.2.9)$$

The receiver decrypts the ciphertext with the following formula:

$$m' = y^d \bmod n \qquad (3.2.2.10)$$

RSA Algorithm: Example

Select p1 = 5 and p2 = 17 (3.2.2.11)

Calculate n = p1 · p2 = 85 (3.2.2.12)

ff. $\varphi(n) = \varphi(85) = \varphi(5 \cdot 17) = \varphi(5) \cdot \varphi(17) = 4 \cdot 16 = 64$ (3.2.2.13)

Select d with $\gcd(d,\varphi(n)) = \gcd(d,64) = 1$ (3.2.2.14)

d = 7 (3.2.2.15)

Select e with $\gcd(d \cdot e,\varphi(n)) = \gcd(7 \cdot e,64) = 1$ (3.2.2.16)

e = 55 (3.2.2.17)

Now make e = 55, n = 85 public and keep d = 7 secret. The sender can now encrypt plaintext using the key e and n. Example:

Plaintext m = 20 (ASCII Code) (3.2.2.18)

$y = 20^e \bmod n$ (3.2.2.19)

$y = 20^{55} \bmod 85 = 45$ (3.2.2.20)

The receiver can now decrypt the ciphertext using key d and n:

Ciphertext y = 45 (3.2.2.21)

$m = 45^d \bmod n$ (3.2.2.22)

$m = 45^7 \bmod 85 = 20$ (3.2.2.23)

3.2.3 Digital Signatures

Public-Key encryption can be used to sign messages. A message is either a plaintext or ciphertext, which doesn't matter for this subchapter, it will just be called message for this part now. A signature authenticates both the sender and the message itself. Which means, that the sender is who he possesses, and the message comes from the right person while it was not modified on its way to the receiver. Messages are signed by encrypting a hash value of the message also called the digest value. The hash value:

H(M) of a message M (3.2.3.1)

is a short bit pattern that is characteristic for the message, which is calculated by a hash function H(). The hash function H(M) calculates a short characteristic bit pattern, see the following figure for an example.

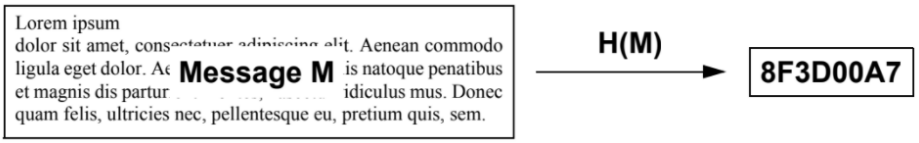


Figure 19, **[F19]**: Cryptographic Hash Functions

As already covered, the requirements for cryptographic hash functions are required to be:

Easy computable

One-way function



Figure 20, **[F20]**: One-way functions shouldn't be reversible easily

Therefore, this means that there should be no easy $H^{-1}(M)$ for an existing H(M). Which generally speaking means that even with unlimited computing power, the time needed would exceed the benefit from cracking the one-way function to retrieve the message.

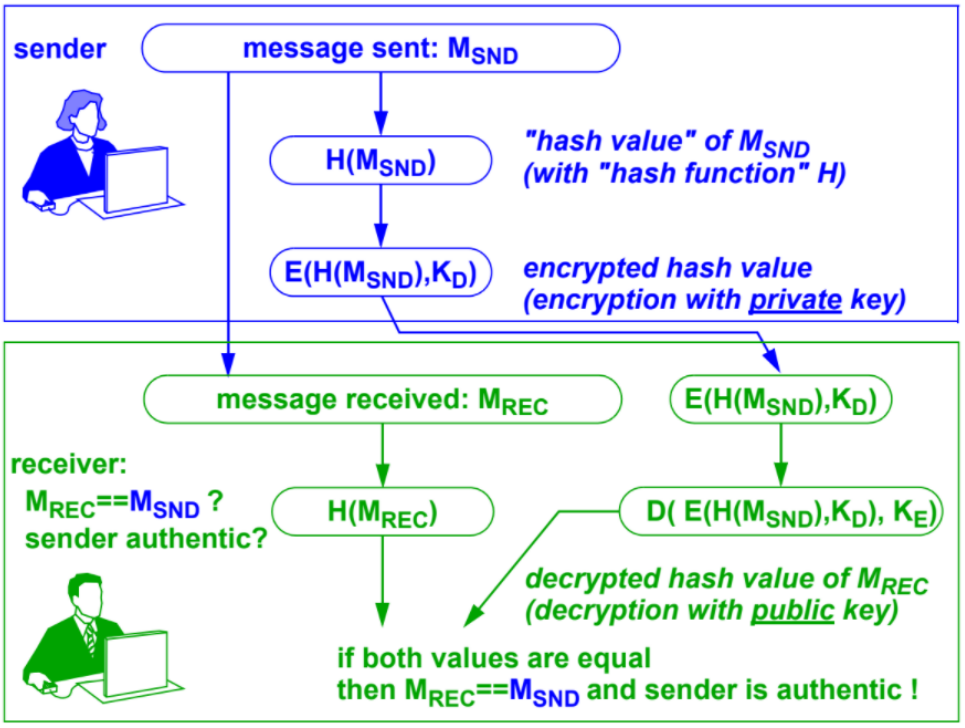


Figure 21, **[F21]**: Digital Signature: Steps

Known cryptographic hash functions in practice are for example Message-Digest Algorithm 5 (MD5, **[A15]**) or the Secure Hash Algorithm (SHA, **[A16]**). Briefly explained, MD5 is a sequence of logical operations and shifts which iteratively on 512-bit blocks of the message result in a 128-bit digest. While SHA is a sequence of logical operations and shifts on blocks of the message which result in a digest of either 224, 256, 384 or 512-bits. However, MD5 is not considered secure anymore, since it's not collision resistant and should be avoided in some, if possible, in all real-world applications, especially in message signing.

But what does collision resistance exactly mean?

It means, that it's hard to find two messages:

$$M_1 \neq M_2 \tag{3.2.3.2}$$

such that:

$$H(M_1) = H(M_2) \tag{3.2.3.3}$$

Wait. This means that collision resistance is crucial for cryptographic hash functions and should be a core part of one-way function algorithm development. Let's take a look at an MD5 example of two strings in hexadecimal format, which show the (weak) collision resistance (ref. [24]):

d131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f89
55ad340609f4b30283e488832571415a085125e8f7cdc99fd91dbdf280373c5b
d8823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0
e99f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70

and

d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f89
55ad340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5b
d8823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0
e99f33420f577ee8ce54b6708280d1ec69821bcb6a8839396f965ab6ff72a70

Each of these blocks has the MD5 hash:

79054025255fb1a26e4bc422aef54eb4

Your String	222ad131dd02c5e6eec4693d9a0698aff95c2fcab58712467eab4004583eb8fb7f89 55ad340609f4b30283e488832571415a085125e8f7cdc99fd91dbdf280373c5b d8823e3156348f5bae6dacd436c919c6dd53e2b487da03fd02396306d248cda0 e99f33420f577ee8ce54b67080a80d1ec69821bcb6a8839396f9652b6ff72a70
MD5 Hash	79054025255fb1a26e4bc422aef54eb4 Copy
SHA1 Hash	71ca11b335e8fc4c0845b81a8f70ff7a29d78e7d Copy
Your String	d131dd02c5e6eec4693d9a0698aff95c2fcab50712467eab4004583eb8fb7f89 55ad340609f4b30283e4888325f1415a085125e8f7cdc99fd91dbd7280373c5b d8823e3156348f5bae6dacd436c919c6dd53e23487da03fd02396306d248cda0 e99f33420f577ee8ce54b67080280d1ec69821bcb6a8839396f965ab6ff72a70
MD5 Hash	79054025255fb1a26e4bc422aef54eb4 Copy
SHA1 Hash	36c4c86838cf5ba6a50544591f3380374eebd2ad Copy

Figure 22, **[F22]**: MD5 Collision demo visualized

It is given that cryptographic hash functions should generate a total new digest from small changes otherwise considered to be insecure.

3.2.4 Steganography

Steganography represents the art of concealing information in a bulk of open information, typically an image file, but it can be anything else like an executable or word document, etc. The limit of this technique is your own fantasy, since data can be hidden almost anywhere in electronic systems, which offers massive potential. This way, eves can hardly find out if and where there is hidden information. I will merely demonstrate a brief example of hidden information in the low-order bits of the pixels of an image file to store secret information and won't make a big deal out of this one, since it's not that relevant for password-less authentication but an interesting topic also.

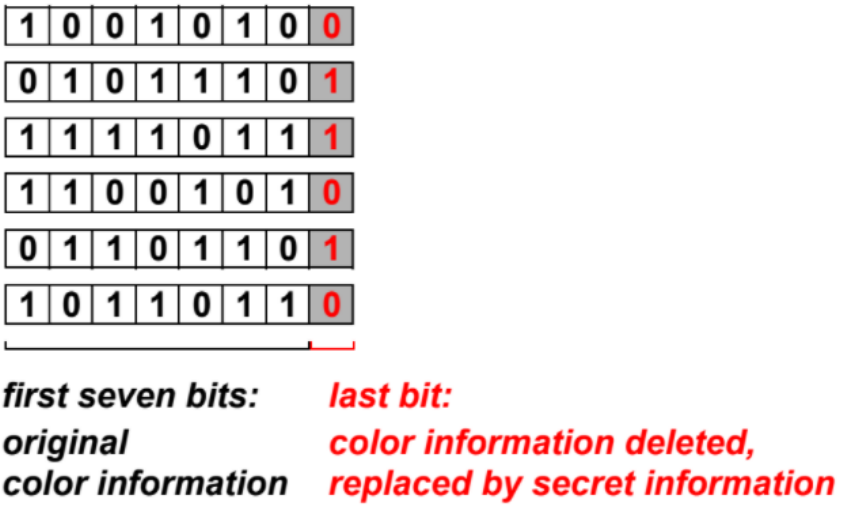


Figure 23, **[F23]**: Manipulating the low-order bits of an image file and hiding information inside the pixels

In the figure above you are seeing that we concealed the information in the last bit of each pixel, which results in a minimal change of the actual picture, however this should be invisible to the normal user, so it won't be recognized. The intended recipient will however understand the information, since he likely knows the exact position of the information to check in the image.

3.3 Key Distribution



Figure 24, **[F24]**: Cryptographic key visualized in binary format

Key Distribution, also often called key exchange in literature, is in core the transfer of keys between communication partners. This is a critical part of cryptography. Hence, key distribution must be secure to enable secure encrypted communication using these keys. This means that secret keys must remain secret and public keys have to be authentic. Authentic in the sense of the sender being who he possesses to be and the message not being modified on its way by any third party. Therefore, key distribution should not rely on the security of a channel and should be automatized.

To explain the problem of key distribution, let's take a look at symmetric key cryptography: Both parties have to possess a shared secret key, which they must have exchanged prior to using any encryption. Distribution of these shared secret keys has been problematic, because it involved face-to-face meeting, use of a trusted courier, or delivering the key through an already existing encryption channel, which would then again depend on the security of a previous key exchange which encountered the same above problems.

In public key cryptography, public keys can be distributed freely since they can only encrypt a message and not decrypt. The following sub-chapters are going to cover the famous:

Diffie-Hellman Key Exchange

as a method to securely exchange cryptographic keys over a public channel. This was one of the first public-key protocols in history. We are also going to cover the authentication of public keys, which is a requirement for a variety of authenticated protocols. Note that, Diffie-Hellman key agreement itself is a non-authenticated key-agreement protocol, but provides the basis for these.

3.3.1 Diffie-Hellman Key Exchange

The most important thing about the Diffie-Hellman Key Exchange is that you're not sharing information during the key exchange, you're creating a key together. Which is a way of generating a shared secret between two people in such a way that the secret can't be seen by observing the communication. This, in particular, is useful, because this allows implementation of technology which allows two parties to exchange encrypted messages on top of a non-secured channel.

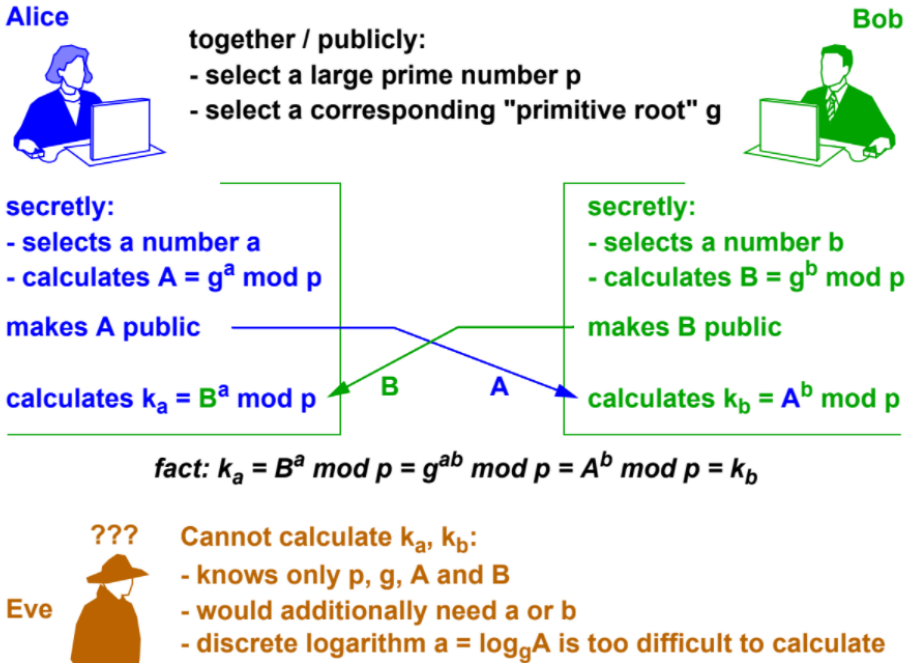


Figure 25, **[F25]**: The Steps of the Diffie-Hellman Protocol

The figure above illustrates the steps of Diffie-Hellman in detail. Even in case the traffic is recorded and later analyzed, there's absolutely no way to figure out what the key was, even though the exchanges that created it may have been visible. This is where:

perfect forward secrecy

comes from. Nobody analyzing the traffic at a later date can break in because the key was never saved, never transmitted and never made visible anywhere. The main reason this is secure is that, unlike normal log(x), the modular log(x) is thought to be hard to compute and would take too long to calculate in order to be effective.

3.3.2 Authentication of Public Keys

Public-Keys for security reasons should be authenticated. There are multiple possible Dangers which public-keys unsecured channel communication could theoretically possess, like:

Man-in-the-middle attacks

Which could be executed, and a potential Mallory would interfere with the public-key protocol and sneak in his own key K_m, pretending it to be Alice's one, for example. This way he would be able to read the encrypted transmissions later on.

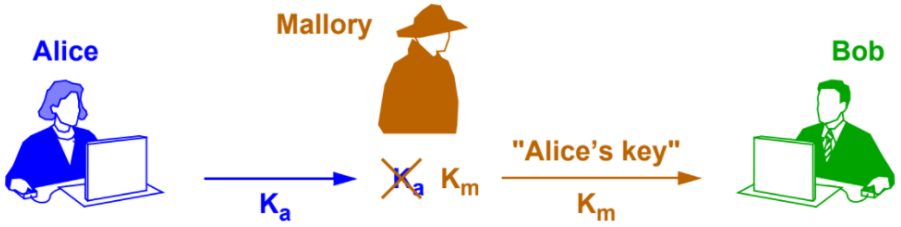


Figure 26, **[F26]**: Man-in-the-middle attack visualized

The solution to such an attack is that we can use a Certification Authority (CA, **[A17]**) / Public Key Infrastructure (PKI, **[A18]**). The PKI registers and distributes trustworthy keys. An alternative to CAs is represented by the Web of Trust. In this concept, Alice's key is signed by many other users. The more users, the more trustworthy.

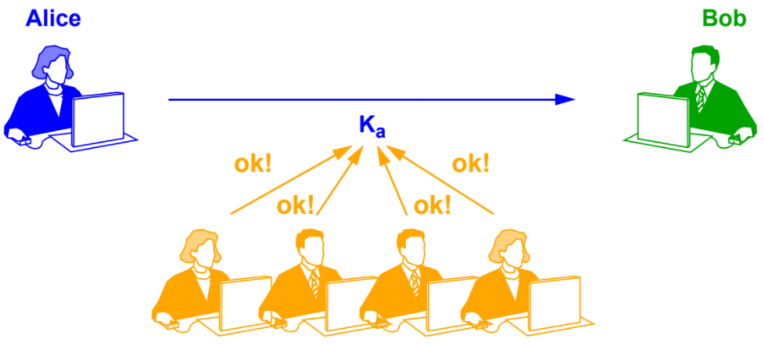


Figure 27, **[F27]**: Web of Trust visualized

Since using CAs is the more common way of authenticating public keys, the Web of Trust is only presented to show another possibility. We will set the focus on using CAs. Typically, the public key of the top-most CA is well-known and stored in the Web browser, so you can always verify these. This, however, requires your web browsers to not contain any malicious injection software, which could change this key tresor. A faulty key tresor may lead to faulty CAs which, in turn, could lead to fake sites stealing your data.

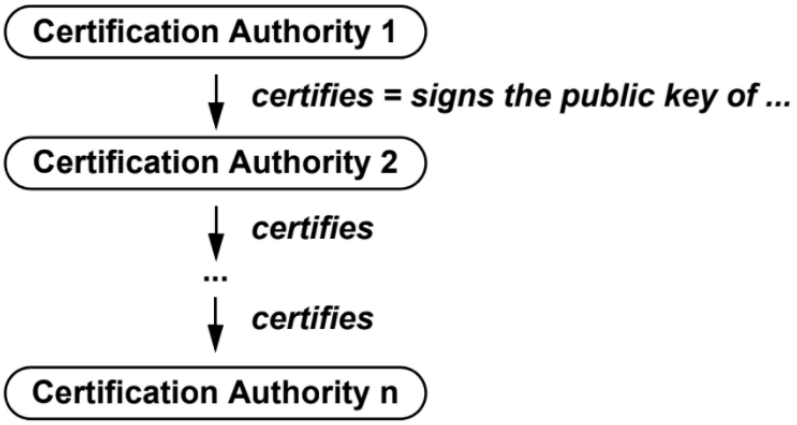


Figure 28, **[F28]**: Hierarchy of Certification Authorities

Public Key Infrastructure is required for activities, where simple passwords are an inadequate authentication method and a more rigorous proof of identity is required to confirm the parties involved in the communication and to validate the information / messages being transferred.

3.4 Transactions

Transactions are the core-part of processing information in computer science. These transactions are divided into individual, indivisible operations. Each transaction must succeed or fail as a complete unit — it can never be only partially complete. Therefore, the definition for a Transaction T is:

$$T = \{ r \mid r = \text{success } s \text{ or } r = \text{failed } f \} \tag{3.4.1}$$

Transactions do not create new transactions. They do have a definite beginning and an end. Therefore, we can say that a transaction is finite. In contrast to processes, which can create new processes, they have a definite beginning, but they might not have an end. The main difference between transactions and processes is the way they communicate with each other. Transactions can only communicate indirectly with each other via the results of tasks stored in the database. Processes can directly communicate with each other, as well as indirectly with each other via side effects on the file system (for example). It's important to know this, since transactions for themselves can't execute tasks alone — they are only a block of commands or data.

Main characteristics of transactions therefore are as following:

T is not a process in an Operating System

Mathematical models / structures are behind T

In good systems, transaction processing is respected by:

A Atomicity

C Consistency

I Isolation

D Durability

(ACID, **[A19]**)

To make transaction processing smooth and not prone to errors. Atomicity is the concept that the components of a transaction are atomic — meaning indivisible. The whole transaction as one must succeed or fail. Consistency does ensure that an illegal transaction is rejected, so that the integrity of the database is upheld. Isolation will ensure that each transaction will be isolated and dealt with in a way that does not affect others. While durability ensures that data is saved once a transaction has been completed. Even if there is a hardware failure immediately after a transaction, the data will be safe.

4 FIDO Alliance Specifications

FIDO Alliance is enabling modern authentication standards to help reduce the world's over-reliance on passwords and other shared secrets. Providing a login experience that is more secure than passwords and SMS 2FA. Simpler for consumers, and easier for service providers to deploy and maintain. They are driven by hundreds of global tech leaders such as Microsoft, Google and Visa. For this purpose, the FIDO Alliance has published three sets of specifications in total for now. These sets are used for simpler, stronger user authentication. The sets are as following:

FIDO Universal Second Factor (FIDO U2F, **[A20]**)

FIDO Universal Authentication Framework (FIDO UAF, **[A21]**)

Client to Authenticator Protocols (CTAP, **[A22]**)

All the above-mentioned FIDO protocols are based on public key cryptography and are strongly resistant to phishing, as already covered in this thesis beforehand. By now, you should know what this term means, hopefully. The possible use cases and deployment scenarios will also be covered as part of the subsequent chapters.

Key benefits of using FIDO include encrypted transmissions, improved user experience and reducing the strain on the relying parties' servers, cutting costs and conserving operational resources.

Why should I even bother with FIDO?

FIDO specifications have been peer-reviewed by many of the world's top public- and private-sector security experts over a more extensive period now. New authenticators are being introduced nearly daily, and they're written to the FIDO specifications, which makes FIDO Certified deployments future-proof, while direct integrations with an operating system would need to be perpetually reworked as the market keeps changing. In addition, FIDO specifications allow for users to select between several biometric authentication methods (face, voice, fingerprint, …), whereas native device authenticators typically force users to a single modality, which in turn decreases security and convenience. Last not least, in the event a certain biometric modality is ever compromised, FIDO deployments can instantly switch to an alternate authentication method or layer several biometrics together for enhanced security.

4.1 Universal Second Factor

FIDO U2F supports the use of a strong second factor, such as a FIDO security key. It represents an open standard that strengthens and simplifies 2FA using specialized Universal Serial Bus (USB, **[A23]**) or near-field communication (NFC, **[A24]**) devices based on similar security technology as found in smart cards. Its predecessor is the FIDO2 project, which is covered under the CTAP specification later on.

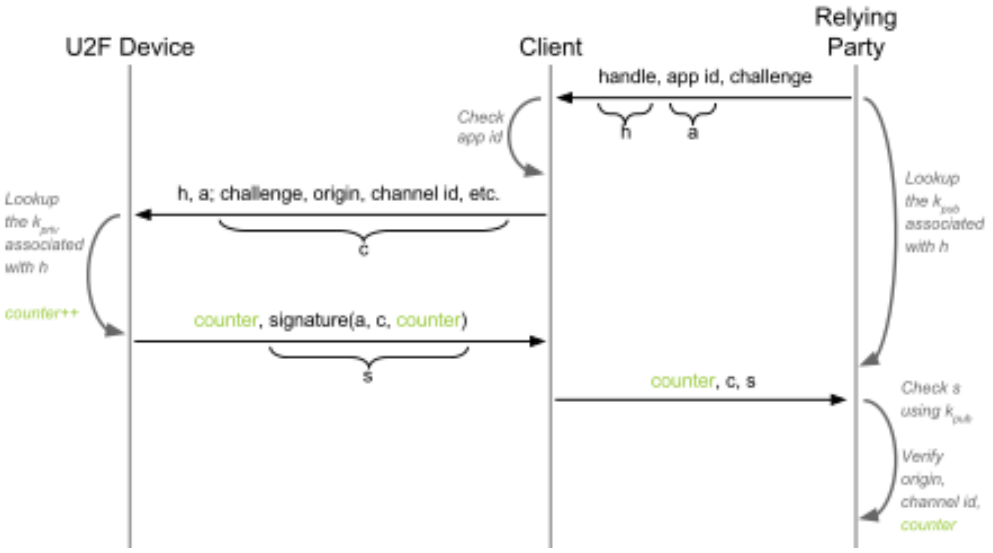


Figure 29, **[F29]**: FIDO U2F authentication workflow

As shown in the figure above, FIDO U2F, simply said, is no more than adding an additional security layer using a hardware token. The client is sending a login or transaction request to the relying party which has the public key from the registration of the hardware token, note that the private key never left the U2F device, then authenticates it using the U2F device. It is required that the used hardware token is FIDO certified, but since FIDO has the highest market share, this shouldn't be a big deal.

4.2 Universal Authentication Framework

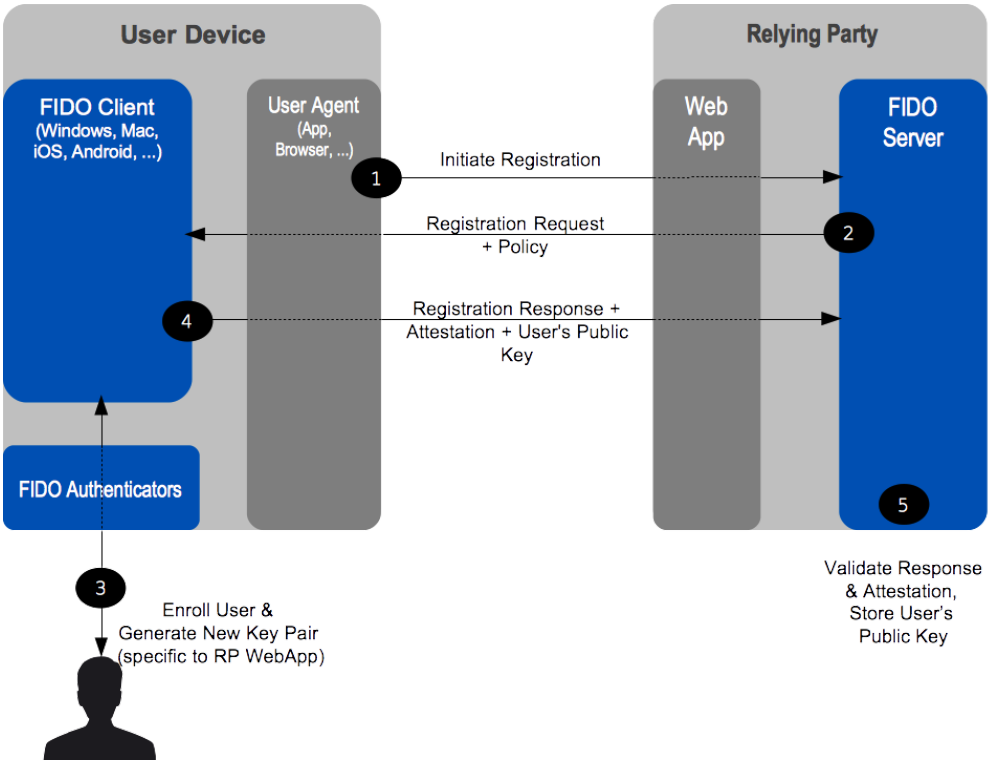


Figure 30, **[F30]**: FIDO UAF registration message flow

FIDO UAF supports passwordless authentication experiences, commonly utilizing a mobile device's biometric capabilities. The network protocol regulates the procedure for proving access authorization for web-based services. The user must register with an online service and can then log in to it locally with his device as often as desired. The device itself is using the K_{priv} which was created on registration to authenticate the user by his biometrics or other given password with the FIDO Server in the background.

The figure above shows the example registration workflow, where a user registers on a relying party. Therefore, creating a K_{priv} and K_{pub} on the FIDO Authenticator side and sending the K_{pub} as a verified key to the FIDO Server, using public key cryptography.

Following registration, the FIDO UAF Authenticator will be subsequently employed whenever the user authenticates with the relying party or website. In case the authenticator is not present, the relying party can implement various fallback strategies for those occasions. These can range from allowing conventional login with credentials up to disallowing the login.

4.3 Client to Authenticator Protocols

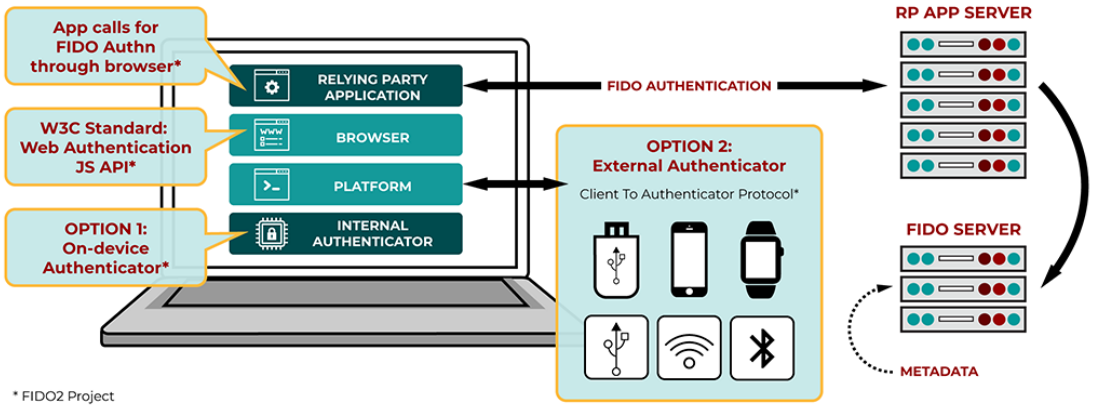


Figure 31, **[F31]**: CTAP authentication overview

CTAP is complementary to the W3C's Web Authentication specification — as one, they are representing the FIDO2 standard. FIDO2 represents the currentest FIDO specification, with FIDO2, the advantages of FIDO are now available in web browsers such as Microsoft Edge, Mozilla Firefox and Google Chrome. It is complementary to UAF, which is nevertheless required for the rich mobile application channel.

CTAP1 is the recent name for FIDO U2F. It defines how to establish communications between FIDO2-enabled browsers and operating systems and a FIDO U2F device, to enable a second-factor authentication experience.

CTAP2 defines how to establish communication between FIDO2-enabled browsers and operating systems and external authenticators (FIDO Security Keys, mobile devices, …) to enable a passwordless, second-factor or multi-factor authentication experience.

The general flow of communications defined by CTAP are as following:

Application establishes a connection with the authenticator

Application gets information about the authenticator and its capabilities

Application delivers a command for an operation that the authenticator supports

Authenticator replies with response data or error

5 From inconvenient to convenient

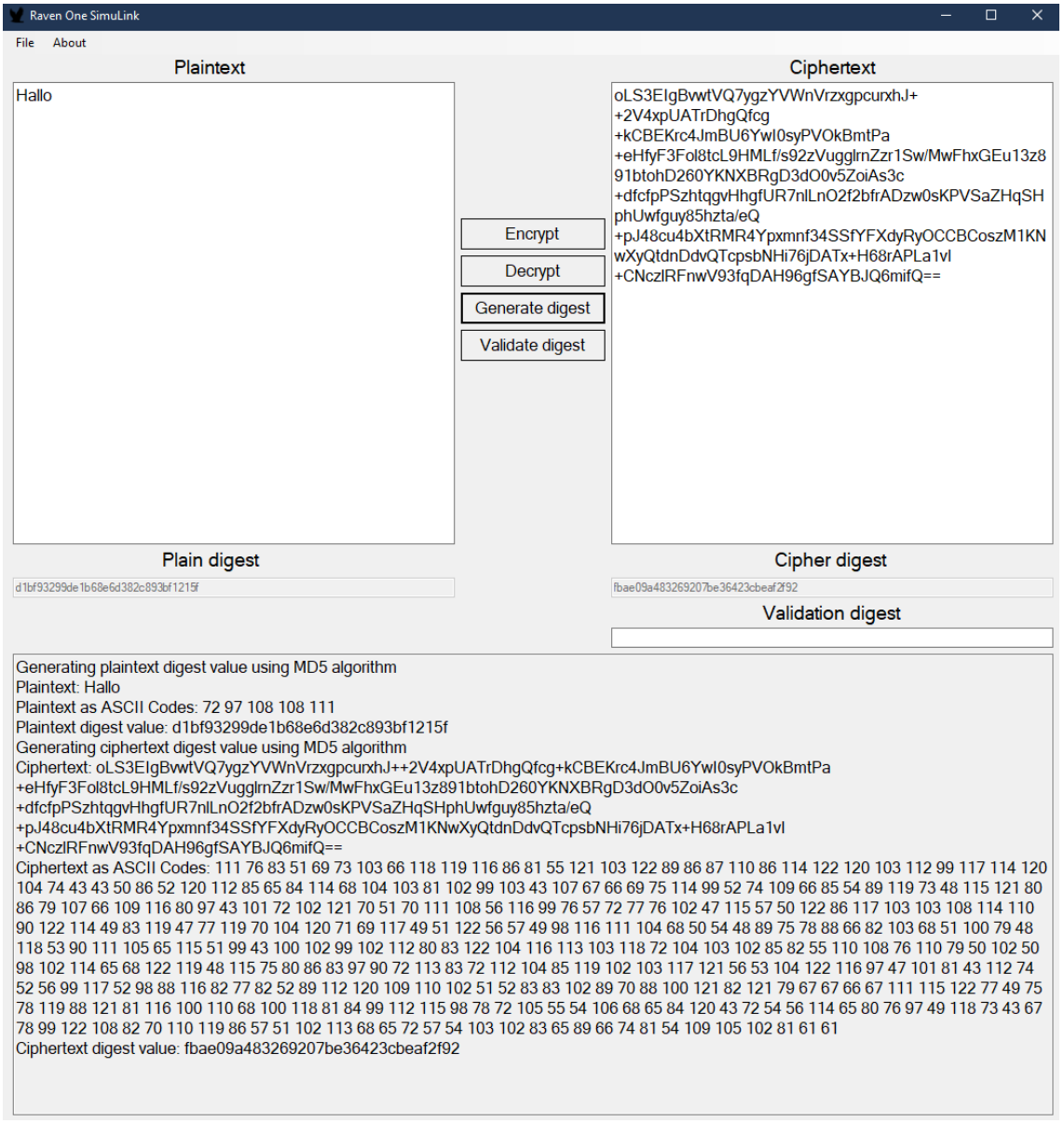


Figure 32, **[F32]**: Raven One SimuLink overview

This chapter will cover Raven One SimuLink, Jamestown use-cases of passwordless approaches and this especially covers the Online Beitritt including the Kundenportal which are both potential candidates for a use-case. The Raven One SimuLink source code can be viewed either in the Appendix chapter or in the corresponding Appendix file which was handed out with this thesis.

Raven One SimuLink is a small simulation software, which was authored by me in C# using the .NET Framework 4.7.2 as Windows Application. This software is used to demonstrate exemplary cryptographic algorithms, respectfully MD5 and RSA. Which uses a graphical user interface (GUI, **[A25]**) to showcase the steps of the cryptographic algorithm and leaves the user a small parametrizability. You can encrypt and decrypt messages using RSA and at the same time generate a digital signature using the SimuLink for your generated messages. Validating hashes and therefore messages can also be done using the Validate digest button.

The application itself has a small description for each of the algorithms under the Menu navigating to File → Algorithms, which shows the high-level description, use-case and other useful information about the specific algorithm that was selected.

This means, however, that Raven One SimuLink implements at least the following:

RSA Algorithm, including parameterizable encrypting and decrypting messages

Digital signature generation and verification using the MD5 cryptographic hash function

Both algorithms MD5 and RSA were implemented using the corresponding Request for Comments (RFC, **[A26]**) by the Internet Engineering Task Force (IETF, **[A27]**). To get more information about the implementation, either look up the corresponding RFC or view the specific chapter of this file which reveals more details.

MD5 RFC under tools.ietf.org/html/rfc1321

RSA RFC under tools.ietf.org/html/rfc8017

The algorithms were implemented by myself, not using any prebuilt library and using calls like “encrypt” or “decrypt”. Therefore, this is my main work of this thesis, which I made myself. Implementing a simulation software running exemplary cryptographic algorithms should be sufficient for a bachelor thesis in computer sciences, hopefully. If you came this far, I want to thank you already for reading my work.

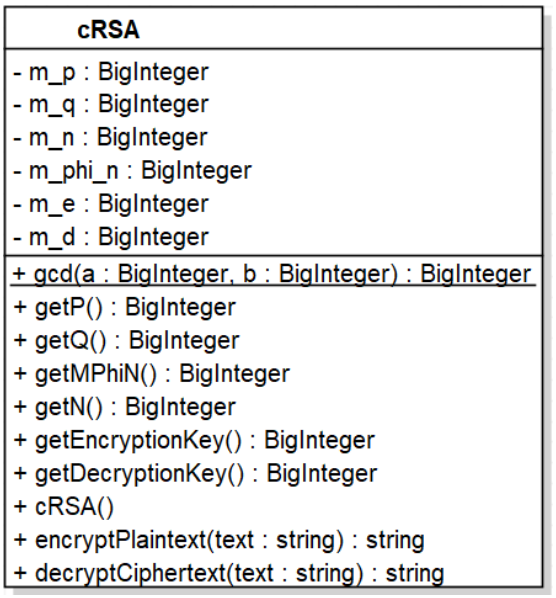


Figure 33, **[F33]**: Raven One SimuLink RSA uml type class diagram

In the figure above and the figure on the following pages, you can view the three main unified modelling language (UML, **[A28]**) class diagrams for the main algorithms and the main gui implemented. The class diagrams have the sole purpose to show off their respective properties and methods implemented. In order to view a workflow of a method, refer to the latter pages for their corresponding uml activity diagrams. Due to the simplicity of these diagrams, there won't be any further explanation of the class diagrams itself.

The class diagrams were created in the free tool called violetumleditor, which is a Java application and should be runnable on most devices. Either, you can use the violetumleditor-2.0.1.jar in the Appendix of this thesis to view and edit them, or download a jar-file yourself from the following trusted source:

violetumleditor under sourceforge.net/projects/violet

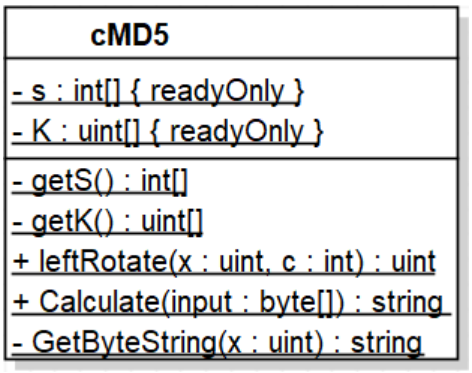


Figure 34, **[F34]**: Raven One SimuLink MD5 uml type class diagram

While the implemented RSA algorithm is used to encrypt and decrypt messages, the SimuLink software will also implement the MD5 algorithm to showcase its functionality of generating hashes to the corresponding messages and also be able to validate these messages by generating the digest value of it. This digest value of a message can be used to check if it was compromised while it was in its simulated transmission channel. These transmission channels in real-world applications are usually treated to be as insecure, therefore the hash values should be compared against each other.

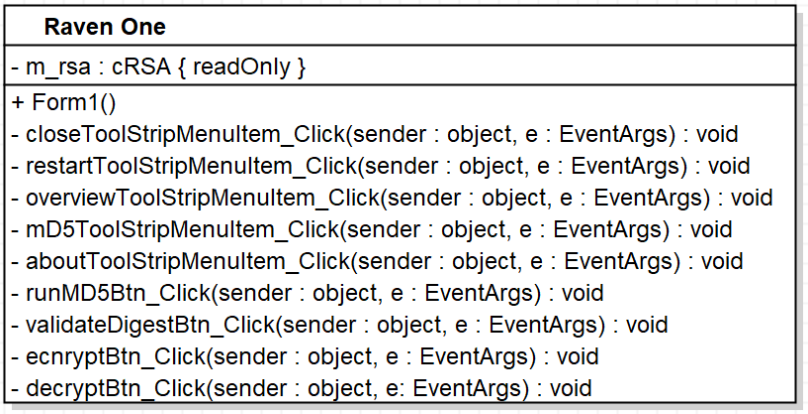


Figure 35, **[F35]**: Raven One SimuLink main gui uml type class diagram

This class represents the switching center of the Raven One SimuLink software. It's used to control and connect both the cMD5 and cRSA classes to represent the software as is. In theory, however, you could still use both classes independent of each other, but to make the simulation work it is suggested to use the main gui connector.

The validation of two digest values is solely done in the main gui connector and not dependent on any class. This means that, both digest values are character for character compared with each other. There is no other special option done performing the validateDigestBtn_Click(...).

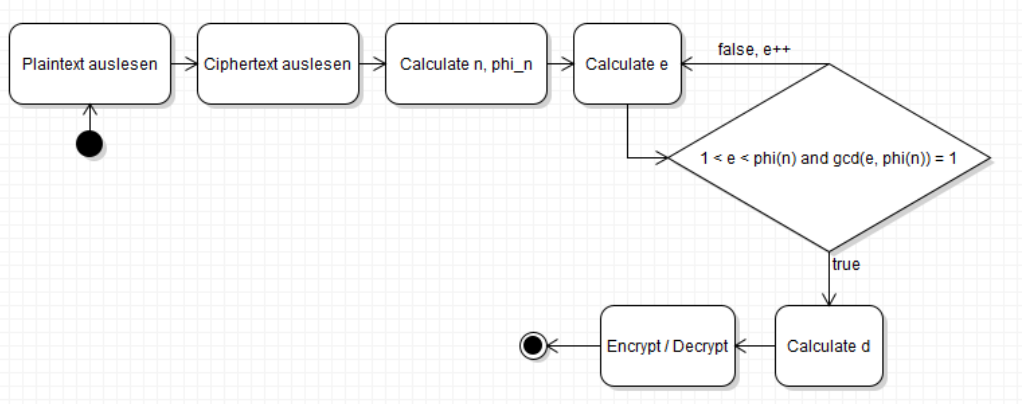


Figure 36, **[F36]**: Raven One SimuLink RSA uml type activity diagram

Activity diagrams are graphical representations of workflows, stepwise activities or actions in computer science. For Raven One SimuLink, there are two main workflows also called algorithms — RSA and MD5, which are described as uml type activity diagrams.

The RSA algorithm first selects two large prime numbers, **p** and **q**. These two large distinct prime numbers are constants in the SimuLink software, for testing purposes. The prime numbers need to be large in real-world appliances, so that they will be difficult for someone to figure out. The next step in SimuLink is, calculating:

$$n = p * q \tag{5.1}$$

$$phi_n = (p - 1) * (q - 1) \tag{5.2}$$

Now we get to the interesting part, selecting the encryption key. We shall select e, such that e is co-prime to phi_n and $1 < e < phi_n$. The pair of numbers (n, e) makes up the public key, which can be freely shared. Two decimals are co-prime if the only positive integer that divides them is 1. Finally, we need to calculate the decryption key. Calculate d such that:

$$d = 1 \bmod phi_n \tag{5.3}$$

The decryption key can be found using the extended euclidean algorithm - gcd. The pair (n, d) makes up the private key, which has to be kept secret. Encryption as following:

$$C = P^e \bmod n \tag{5.4}$$

Decryption as following:

$$P = C^d \bmod n \tag{5.5}$$

Note: mod stands for modulus also known as %-operator.

The main MD5 algorithm does operate on a 128-bit state, divided into four 32-bit words, which are represented by A, B, C and D in source code. These are initialized to fixed constants in hex, which are as following:

A = 0x67452301; B = 0xefcdab89; C = 0x98badcfe; D = 0x10325476;

The main algorithm then uses each 512-bit message block in turn to modify the state. The processing of a message block consists of four similar stages, termed **rounds**. Each round is composed of 16 similar operations based on a nonlinear function F, modular addition, and left rotation. A 512-bit message block consists of four times a 128-bit block using 16 similar operations. The mentioned 16 similar operations times four and saved in the variable s, which is initialized with the following values:

7, 12, 17, 22,

5, 9, 14, 20,

4, 11, 16, 23,

6, 10, 15, 21

K is calculated using the binary integer part of the sines of integers (radians) as constants. The following four functions are possible to be used in each round:

F(**B**, **C**, **D**) = (**B** AND **C**) OR (NOT **B** AND **D**)

G(**B**, **C**, **D**) = (**B** AND **D**) OR (**C** AND NOT **D**)

H(**B**, **C**, **D**) = **B** XOR **C** XOR **D**

I(**B**, **C**, **D**) = **C** XOR (**B** OR NOT **D**)

5.1 Jamestown Online

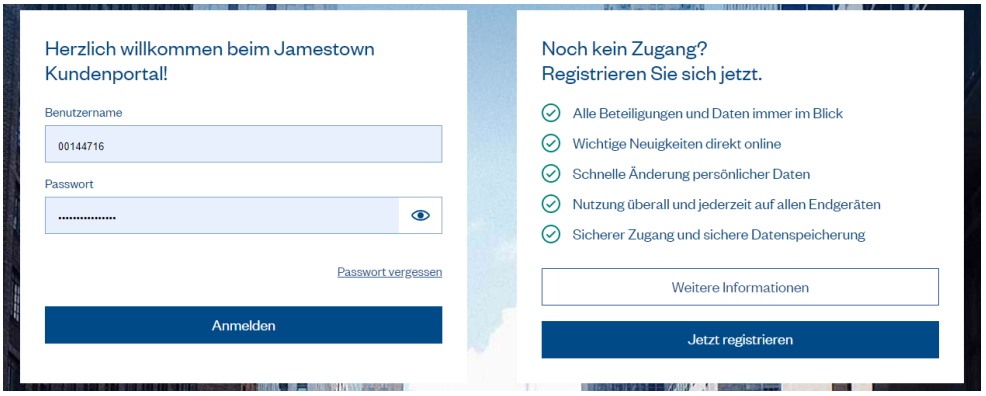


Figure 37, **[F37]**: Jamestown Online Kundenportal overview

In the figure above you can view the already existing but steadily improving Online Kundenportal under online.jamestown.de which on the left side offers a traditional login procedure and a registration procedure on the right side. This portal is important for this thesis, because it offers a couple of possible use-cases for passwordless authentication. On the following pages we will find out about the traditional procedures and how we can improve them by making these passwordless on a high-level description.

Modern methods usually come with restrictions, such as passwordless does as well. Users won't be able to use this feature without a sufficient modern smartphone, for example, which will be a reason why conventional authentication methods won't disappear for a longer time period. This would cut these users out of modern architecture systems and force them to adapt, while they possibly don't want this step.

Jamestown has already had interviews with a couple of service providers to realize passwordless authentication organization-wide, like NEVIS Security GmbH. As of today, we are already in a testing stage using FIDO hardware tokens, but this testing stage is not going to be relevant for this thesis, nevertheless.

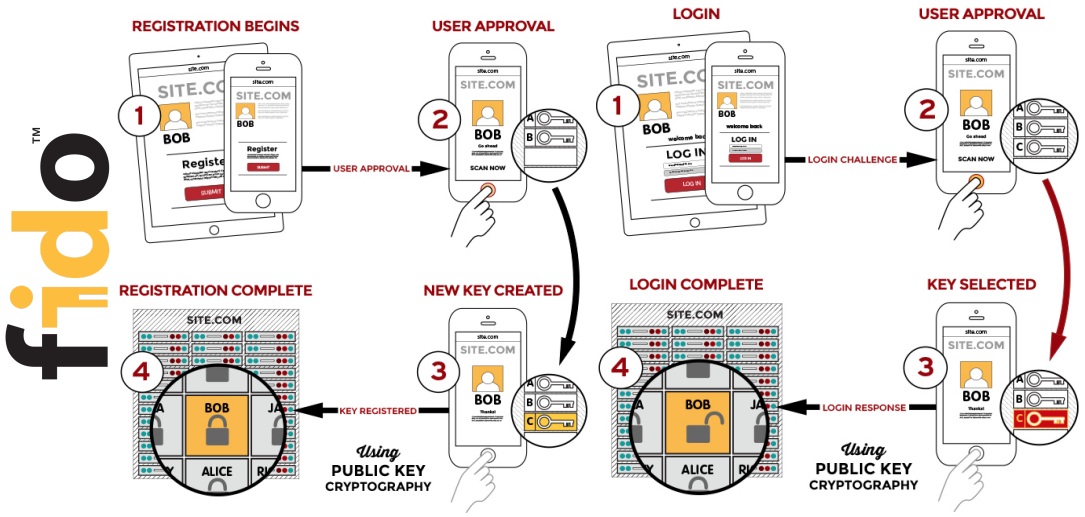


Figure 38, **[F38]**: FIDO passwordless collage

In the figure above, you can view the process of FIDO while registering and logging in a user. This enables passwordless authentication, basically. You are forced to register a unique trait, which is a fingerprint in this case, since these are most widely spread they make up a good example. This fingerprint is not stored anywhere else except on the smartphone itself. Usually, smartphones also have inbuilt biometric tresors, which should be relatively secure. In this case, the fingerprint is your private-key while the public-key is transmitted to FIDO servers, used to authenticate on login procedures. This means that no one has to worry about a possible data breach on FIDO or anywhere, because the only valuable data is the private-key. This private-key, however, is solely stored on your local devices.

In conclusion, we can say that this technique should be sufficient for Jamestown-Online Kundenportal, and it's Online-Beitritt.

5.2 Online-Zeichnungsschein

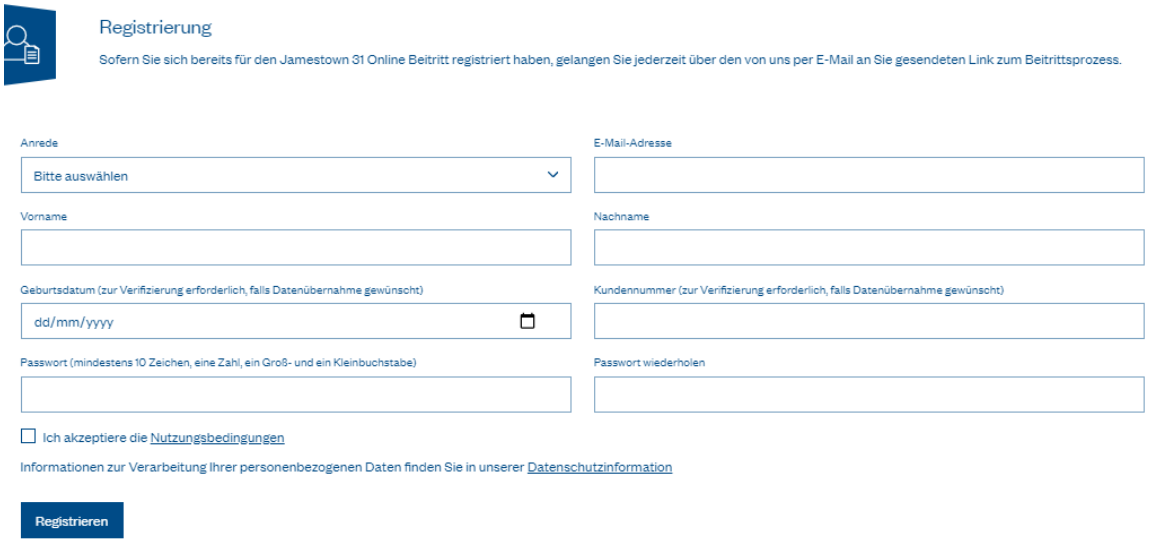


Figure 39, **[F39]**: Jamestown Online-Beitritt Zeichnungsschein overview

Jamestown Online-Zeichnungsschein, however, is another web application which has the potential to benefit from the passwordless authentication at least a bit in the future. For the sole purpose of logging in, while filling the form until submitted or making an increasing charge, this convenient feature could be handy. Allowing users more interaction with the form when implementing a more convenient access routine.

As the service of passwordless login should be reliable and tested, implementing this feature into the Online-Zeichnungsschein would be a later step in the planning. First it has to be tested in the Kundenportal, because the Online-Zeichnungsschein allows customers to invest large funds, which is a high security appliance and should be treated as one at any time.

6 Conclusion

In conclusion, we can say that offering modern authentication is a must-have for companies nowadays. It's worth the effort to realize this, because most customers want fancy and convenient authentication but at the same time rely on high security when making large investments, which has to be somehow compensated by the company offering these features. At this point of time, MFA — also called password-less authentication comes handy. Allowing users to login without having to enter a password anymore in the normal case will make them feel more comfortable and in return may generate a higher revenue or closer customer contact, which could make the business relationships last longer.

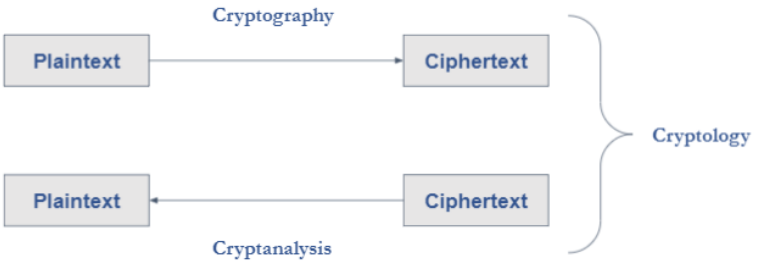


Figure 40, **[F40]**: Cryptology definition

The covered algorithms for cryptographic functions like RSA, AES or MAC and the Raven One SimuLink Software do indeed show on a low-level base of software, that it's possible and secure to enable these modern algorithms. Data Protection in Europe is already a really sensitive topic, which should be taken seriously. There should be no company sharing data in unsecured channels without proper encryption methods. Data Privacy is increasingly threatened by companies like Google, Facebook, Microsoft, NSA and governments, only to name some, but these are probably the most notable ones. That's why these countermeasures like E-Mail encryption, Secure Internet access or even Anonymous Internet surfing should be applied.

This thesis could be the pre-work for a potential master thesis, covering the basic needs of cryptographic topics. The master thesis would then build upon these cryptographic information given here and intensify this, additional mathematical problems would be covered, which could raise complexity so that quantum computers won't be able to break this encryption type anymore with their impressive computational power — making the algorithms post-quantum proof. However, this is just an idea which can transition into a lot of different things, since cryptography (including post-quantum cryptography) is a really broad topic which seems like it has no borders.

7 Appendix

I. cMD5.cs

```
/*
* TH Köln
* Institut für Nachrichtentechnik (INT)
*
* Raven One SimuLink
* cMD5.cs
* MD5 Class definition written in C#
*
* Author: Mert Torun
*
* Date: 2021-04 - 2021-06
*
* This program is free software: you can redistribute it and/or modify
* it under the terms of the GNU General Public License as published by
* the Free Software Foundation, either version 3 of the License, or
* (at your option) any later version.
*
* This program is distributed in the hope that it will be useful,
* but WITHOUT ANY WARRANTY; without even the implied warranty of
* MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
* GNU General Public License for more details.
*/
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;

namespace Raven_One_SimuLink
{
    /// <summary>
    /// RFC for MD5 at https://tools.ietf.org/html/rfc1321
    /// </summary>
    class cMD5
    {
        // Round shift values
        // Whereas the variable s specifies the per-round shift amounts
        static readonly int[] s = new int[64]
        {
            7, 12, 17, 22,  7, 12, 17, 22,  7, 12, 17, 22,  7, 12, 17, 22,
            5,  9, 14, 20,  5,  9, 14, 20,  5,  9, 14, 20,  5,  9, 14, 20,
            4, 11, 16, 23,  4, 11, 16, 23,  4, 11, 16, 23,  4, 11, 16, 23,
            6, 10, 15, 21,  6, 10, 15, 21,  6, 10, 15, 21,  6, 10, 15, 21
        };

        public static int[] getS()
        {
            return s;
        }
```

```
// Constant K Values
// Calculated using binary integer part of the sines of integers (Radians) as constants
// for i from 0 to 63 do
//  K[i] := floor(232 × abs (sin(i + 1)))
// end for
static readonly uint[] K = new uint[64]
{
   0xd76aa478, 0xe8c7b756, 0x242070db, 0xc1bdceee,
   0xf57c0faf, 0x4787c62a, 0xa8304613, 0xfd469501,
   0x698098d8, 0x8b44f7af, 0xffff5bb1, 0x895cd7be,
   0x6b901122, 0xfd987193, 0xa679438e, 0x49b40821,
   0xf61e2562, 0xc040b340, 0x265e5a51, 0xe9b6c7aa,
   0xd62f105d, 0x02441453, 0xd8a1e681, 0xe7d3fbc8,
   0x21e1cde6, 0xc33707d6, 0xf4d50d87, 0x455a14ed,
   0xa9e3e905, 0xfcefa3f8, 0x676f02d9, 0x8d2a4c8a,
   0xfffa3942, 0x8771f681, 0x6d9d6122, 0xfde5380c,
   0xa4beea44, 0x4bdecfa9, 0xf6bb4b60, 0xbebfbc70,
   0x289b7ec6, 0xeaa127fa, 0xd4ef3085, 0x04881d05,
   0xd9d4d039, 0xe6db99e5, 0x1fa27cf8, 0xc4ac5665,
   0xf4292244, 0x432aff97, 0xab9423a7, 0xfc93a039,
   0x655b59c3, 0x8f0ccc92, 0xffeff47d, 0x85845dd1,
   0x6fa87e4f, 0xfe2ce6e0, 0xa3014314, 0x4e0811a1,
   0xf7537e82, 0xbd3af235, 0x2ad7d2bb, 0xeb86d391
};

public static uint[] getK()
{
   return K;
}

// leftRotate using bit rotation, known as circular shift rotation
// Circular shift shifts the characters to the left by an x amount, appending
// the extra ones back at the end
public static uint leftRotate(uint x, int c)
{
   return (x << c) | (x >> (32 - c));
}

// assumes whole bytes as input
public static string Calculate(byte[] input)
{
   uint a0 = 0x67452301; // A
   uint b0 = 0xefcdab89; // B
   uint c0 = 0x98badcfe; // C
   uint d0 = 0x10325476; // D
   var addLength = (56 - ((input.Length + 1) % 64)) % 64; // calculates the new length with padding
   var processedInput = new byte[input.Length + 1 + addLength + 8];
   Array.Copy(input, processedInput, input.Length);
   processedInput[input.Length] = 0x80; // adds 1 (0x80 = 128 in dec.)

   // Little Endian vs Big Endian
   // LSB = Least-significant byte
   // MSB = Most-significant byte
   // LSB > MSB     vs MSB > LSB
```

```
// 78 56 34 12  vs 12 34 56 78

byte[] length = BitConverter.GetBytes(input.Length * 8); // bitconverter returns as little-endian
Array.Copy(length, 0, processedInput, processedInput.Length - 8, 4); // add length in bit

for (int i = 0; i < processedInput.Length / 64; ++i)
{
   // copy the input to variable M
   uint[] M = new uint[16];
   for (int j = 0; j < 16; ++j)
   {
      M[j] = BitConverter.ToUInt32(processedInput, (i * 64) + (j * 4));
   }

   // initialize round variables
   uint A = a0;
   uint B = b0;
   uint C = c0;
   uint D = d0;
   uint F = 0;
   uint g = 0;

   // primary loop
   for (uint k = 0; k < 64; ++k)
   {
      if (k <= 15)
      {
         F = (B & C) | (~B & D);
         g = k;
      }
      else if (k >= 16 && k <= 31)
      {
         F = (D & B) | (~D & C);
         g = ((5 * k) + 1) % 16;
      }
      else if (k >= 32 && k <= 47)
      {
         F = B ^ C ^ D;
         g = ((3 * k) + 5) % 16;
      }
      else if (k >= 48)
      {
         F = C ^ (B | ~D);
         g = (7 * k) % 16;
      }

      var dtemp = D;
      D = C;
      C = B;
      B += leftRotate((A + F + K[k] + M[g]), s[k]);
      A = dtemp;
   }

   a0 += A;
   b0 += B;
```

```
            c0 += C;
            d0 += D;
        }

        return GetByteString(a0) + GetByteString(b0) + GetByteString(c0) + GetByteString(d0);
    }

    // Returns x as string in HEX format
    private static string GetByteString(uint x)
    {
        return String.Join("", BitConverter.GetBytes(x).Select(y => y.ToString("x2")));
    }
  }
}
```

II. cRSA.cs

```
/*
 * TH Köln
 * Institut für Nachrichtentechnik (INT)
 *
 * Raven One SimuLink
 * cRSA.cs
 * RSA Class definition written in C#
 *
 * Author: Mert Torun
 *
 * Date: 2021-04 - 2021-06
 *
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 */
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Numerics;

namespace Raven_One_SimuLink
{
  /// <summary>
  /// RFC for RSA at https://tools.ietf.org/html/rfc8017
  /// </summary>
  class cRSA
  {
    private BigInteger m_p;
```

```
private BigInteger m_q;
private BigInteger m_n; // n = p * q
private BigInteger m_phi_n; // phi(n) = (p-1)*(q-1)
private BigInteger m_e; // Encryption key, public key
private BigInteger m_d; // Decrption key, private key

// Greatest common divisor function implemented using euclidean algorithm, see mathematics for more details
public static BigInteger gcd(BigInteger a, BigInteger b)
{
   while (a != 0 && b != 0)
   {
      if (a > b)
      {
         a %= b;
      }
      else
      {
         b %= a;
      }
   }

   return a | b;
}

public BigInteger getP()
{
   return m_p;
}
public BigInteger getQ()
{
   return m_q;
}
public BigInteger getMPhiN()
{
   return m_phi_n;
}
public BigInteger getN()
{
   return m_n;
}

public BigInteger getEncryptionKey()
{
   return m_e;
}

public BigInteger getDecryptionKey()
{
   return m_d;
}

// Constructor
// p and q have to be large distinct prime numbers (!)
public cRSA()
{
```

```
            m_p = 11;
            m_q = 17;
            m_n = m_p * m_q;
            m_phi_n = (m_p - 1) * (m_q - 1);

            // for checking that 1 < e < phi(n)
            // and gcd(e, phi(n)) = 1
            m_e = 3;
            BigInteger track;
            while (m_e < m_phi_n)
            {
                track = gcd(m_e, m_phi_n);

                if (track == 1)
                {
                    break;
                }
                else
                {
                    m_e++;
                }
            }

            m_d = (1 + (2 * m_phi_n)) / m_e;
        }

        public string encryptPlaintext(string text)
        {
            BigInteger t = Convert.ToInt32(text);
            return Convert.ToString(BigInteger.Pow(t, (int)m_e) % m_n);
        }

        public string decryptCiphertext(string text)
        {
            BigInteger t = Convert.ToInt32(text);
            return Convert.ToString(BigInteger.Pow(t, (int)m_d) % m_n);
        }
    }
}
```

III. Raven One.cs

```
/*
 * TH Köln
 * Institut für Nachrichtentechnik (INT)
 *
 * Raven One SimuLink
 * Raven One.cs
 * Main GUI definition written in C#
 *
 * Author: Mert Torun
 *
 * Date: 2021-04 - 2021-06
 *
```

```
 * This program is free software: you can redistribute it and/or modify
 * it under the terms of the GNU General Public License as published by
 * the Free Software Foundation, either version 3 of the License, or
 * (at your option) any later version.
 *
 * This program is distributed in the hope that it will be useful,
 * but WITHOUT ANY WARRANTY; without even the implied warranty of
 * MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the
 * GNU General Public License for more details.
 */
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using System.Windows.Forms;
using System.Text;

namespace Raven_One_SimuLink
{
   public partial class Form1 : Form
   {
      private readonly cRSA m_rsa;

      public Form1()
      {
         InitializeComponent();
         m_rsa = new cRSA();
      }

      // Menu Strip: File -> Close
      private void closeToolStripMenuItem_Click(object sender, EventArgs e)
      {
         Environment.Exit(0);
      }

      // Menu Strip: File -> Restart
      private void restartToolStripMenuItem_Click(object sender, EventArgs e)
      {
         Application.Restart();
         Environment.Exit(0);
      }

      // Menu Strip: File -> Algorithms -> RSA
      private void overviewToolStripMenuItem_Click(object sender, EventArgs e)
      {
         RSA rsaForm = new RSA();
         rsaForm.Show();
      }

      // Menu Strip: File -> Algorithms -> MD5
      private void mD5ToolStripMenuItem_Click(object sender, EventArgs e)
      {
         MD5 md5Form = new MD5();
         md5Form.Show();
      }
```

```
// Menu Strip: About
private void aboutToolStripMenuItem_Click(object sender, EventArgs e)
{
    About aboutForm = new About();
    aboutForm.Show();
}

// Generate digest button
private void runMD5Btn_Click(object sender, EventArgs e)
{
    plainDigestBox.Clear();
    cipherDigestBox.Clear();
    detailBox.Clear();
    bool ptext = false;
    bool ctext = false;
    cMD5 md5Class = new cMD5();
    string plaintext = plaintextBox.Text;
    string ciphertext = ciphertextBox.Text;

    // Only compute digest if not empty
    if (!String.IsNullOrEmpty(plaintext))
    {
        byte[] plaintextBytes = Encoding.ASCII.GetBytes(plaintext);
        string plainDigest = cMD5.Calculate(plaintextBytes);
        plainDigestBox.Text = plainDigest;
        detailBox.Text += "Generating plaintext digest value using MD5 algorithm"
            + Environment.NewLine + "Plaintext: " + plaintext + Environment.NewLine + "Plaintext as ASCII Codes: ";
        foreach (byte x in plaintextBytes)
        {
            detailBox.Text += x.ToString() + " ";
        }
        detailBox.Text += Environment.NewLine + "Plaintext digest value: " + plainDigest + Environment.NewLine;
        ptext = true;
    }
    if (!String.IsNullOrEmpty(ciphertext))
    {
        byte[] ciphertextBytes = Encoding.ASCII.GetBytes(ciphertext);
        string cipherDigest = cMD5.Calculate(ciphertextBytes);
        cipherDigestBox.Text = cipherDigest;
        detailBox.Text += "Generating ciphertext digest value using MD5 algorithm"
            + Environment.NewLine + "Ciphertext: " + ciphertext + Environment.NewLine + "Ciphertext as ASCII Codes: ";
        foreach (byte x in ciphertextBytes)
        {
            detailBox.Text += x.ToString() + " ";
        }
        detailBox.Text += Environment.NewLine + "Ciphertext digest value: " + cipherDigest;
        ctext = true;
    }

    if (!ptext && !ctext)
    {
        detailBox.Text = "Error, please input plaintext or ciphertext values...";
    }
}
```

```
// Validate digest button
private void validateDigestBtn_Click(object sender, EventArgs e)
{
   detailBox.Clear();
   string cipherDigest = cipherDigestBox.Text;
   string validateDigest = validationDigestBox.Text;

   // Only compute validation if not empty
   if (!String.IsNullOrEmpty(cipherDigest) && !String.IsNullOrEmpty(validateDigest))
   {
      detailBox.Text += "Cipher digest: " + cipherDigest + Environment.NewLine
         + "Validation digest: " + validateDigest + Environment.NewLine;
      byte[] cipherDigestBytes = Encoding.ASCII.GetBytes(cipherDigest);
      byte[] validationDigestBytes = Encoding.ASCII.GetBytes(validateDigest);
      detailBox.Text += "Cipher digest as ASCII codes: ";
      foreach (byte x in cipherDigestBytes)
      {
         detailBox.Text += x.ToString() + " ";
      }
      detailBox.Text += Environment.NewLine + "Validation digest as ASCII codes: ";
      foreach (byte x in validationDigestBytes)
      {
         detailBox.Text += x.ToString() + " ";
      }
      detailBox.Text += Environment.NewLine;
      bool valid = false;
      valid = String.Compare(cipherDigest, validateDigest) == 0 ? true : false;

      if (valid)
      {
         detailBox.Text += "Both digest values are equal";
      }
      else
      {
         detailBox.Text += "Both digest values are not equal";
      }
   }
   else
   {
      detailBox.Text += "Error, please compute valid cipher digest and input validation digest accordingly";
   }
}

// Encrypt button
private void encryptBtn_Click(object sender, EventArgs e)
{
   ciphertextBox.Clear();
   cipherDigestBox.Clear();
   detailBox.Clear();

   ciphertextBox.Text = m_rsa.encryptPlaintext(plaintextBox.Text);

   detailBox.Text +=
      "P-Value: " + m_rsa.getP() + Environment.NewLine
```

```
            + "Q-Value: " + m_rsa.getQ() + Environment.NewLine
            + "N = p * q Value: " + m_rsa.getN() + Environment.NewLine
            + "Phi(n) = (p-1)*(q-1) Value: " + m_rsa.getMPhiN() + Environment.NewLine
            + "Encryption key: " + m_rsa.getEncryptionKey() + Environment.NewLine
            + "Decryption key: " + m_rsa.getDecryptionKey() + Environment.NewLine
            + "Encrypted Value using p^e mod n which is "
            + plaintextBox.Text + "^" + m_rsa.getEncryptionKey() + " % " + m_rsa.getN()
            + Environment.NewLine + "Result: " + ciphertextBox.Text;
        }

        // Decrypt button
        private void decryptBtn_Click(object sender, EventArgs e)
        {
            plaintextBox.Clear();
            plainDigestBox.Clear();
            detailBox.Clear();

            plaintextBox.Text = m_rsa.decryptCiphertext(ciphertextBox.Text);

            detailBox.Text +=
                "P-Value: " + m_rsa.getP() + Environment.NewLine
                + "Q-Value: " + m_rsa.getQ() + Environment.NewLine
                + "N = p * q Value: " + m_rsa.getN() + Environment.NewLine
                + "Phi(n) = (p-1)*(q-1) Value: " + m_rsa.getMPhiN() + Environment.NewLine
                + "Encryption key: " + m_rsa.getEncryptionKey() + Environment.NewLine
                + "Decryption key: " + m_rsa.getDecryptionKey() + Environment.NewLine
                + "Decrypted Value using c^d mod n which is "
                + ciphertextBox.Text + "^" + m_rsa.getDecryptionKey() + " % " + m_rsa.getN()
                + Environment.NewLine + "Result: " + plaintextBox.Text;
        }
    }
}
```

8 List of Abbreviations

The following describes the significance of various abbreviations and acronyms used throughout the thesis. The page on which each one is defined or first used is also displayed aligned on the right side.

[A1]	**JTO**	Jamestown Online	2
[A2]	**FIDO**	Fast IDentity Online	2
[A3]	**KVG**	Kapitalverwaltungsgesellschaft	8
[A4]	**BaFin**	Federal Financial Supervisory Authority	8
[A5]	**SEC**	Securities and Exchange Commission	8
[A6]	**PIN**	Personal identification number	12
[A7]	**MFA**	Multi factor authentication	12
[A8]	**SSO**	Single sign on	13
[A9]	**2FA**	Two factor authentication	13
[A10]	**OTP**	One-time pad	14
[A11]	**A**	Alice	17
[A12]	**B**	Bob	17
[A13]	**Eve**	Eavesdropper	17
[A14]	**AES**	Advanced Encryption Standard	22
[A15]	**MD5**	Message-Digest Algorithm 5	30
[A16]	**SHA**	Secure Hash Algorithm	30
[A17]	**CA**	Certification Authority	35
[A18]	**PKI**	Public Key Infrastructure	35
[A19]	**ACID**	Atomicity, Consistency, Isolation, Durability	37
[A20]	**FIDO U2F**	FIDO Universal Second Factor	38
[A21]	**FIDO UAF**	FIDO Universal Authentication Framework	38

[A22] **CTAP** Client to Authenticator Protocols 38

[A23] **USB** Universal Serial Bus 39

[A24] **NFC** Near-field communication 39

[A25] **GUI** Graphical user interface 43

[A26] **RFC** Request for Comments 43

[A27] **IETF** Internet Engineering Task Force 43

[A28] **UML** Unified modelling language 44

9 List of Figures

[F1] *FIDO Alliance and Jamestown corporate identity collage*
FIDO Alliance logo taken from
fidoalliance.org/wp-content/uploads/FIDO_Alliance_tagline.png in April 2021. 5

[F2] *FIDO Alliance members overview cutout*
This image shows off an incomplete overview of the members.
Taken from images.macrumors.com/article-new/2020/02/fido-alliance.jpg in May 2021. 6

[F3] *Ponce City Market, Atlanta* 7

[F4] *Office space in Ponce City Market, Atlanta* 7

[F5] *Jamestown 31 Levi's Plaza, San Francisco* 8

[F6] *Jamestown 31 Garco Mill, South Carolina* 9

[F7] *Possible Cracker Attacks*
Taken from [13] Chapter 9: Security. p. 5. 10

[F8] *Admission versus access control in local security systems*
Taken from [13] Chapter 9: Security. p. 7. 11

[F9] *Ali Baba and the Forty Thieves fairy tale colorized*
Taken from wikipedia.org/wiki/Open_sesame in May 2021. 12

[F10] *Google Titan hardware security keys product line-up 2021*
Taken from cloud.google.com/titan-security-key in May 2021. 13

[F11] *One-time pad*
Taken from halls-of-valhalla.org/beta/articles/one-time-pad,42 in May 2021. 14

[F12] *Biometric Authentication*
Taken from [13] Chapter 9: Security. p. 15. 15

[F13] *Main logical block of a biometric system simplified*
Taken from
upload.wikimedia.org/wikipedia/commons/3/3c/Biometric_system_diagram.png
in May 2021. 16

[F14] *Dictionary attack simplified example*
Taken from [13] Chapter 9: Security. p. 19. 19

[F15] *Fundamental Model visualized*
Taken from [13] Chapter 9: Security. p. 55. 21

[F16] *Steps of AES Encryption*
Taken from [13] Chapter 9: Security. p. 59. 23

[F17] *Public-Key Cryptography whereas Alice represents the receiver and Bob the sender*
Taken from upload.wikimedia.org/wikipedia/commons/thumb/f/f9/Public_key_encryption.svg/1920px-Public_key_encryption.svg.png in May 2021. 25

[F18] *Public-Key — The General Model*
Taken from [13] Chapter 9: Security. p. 63. 26

[F19] *Cryptographic Hash Functions*
Taken from [13] Chapter 9: Security. p. 72. 29

[F20] *One-way functions shouldn't be reversible easily*
Taken from [13] Chapter 9: Security. p. 72. 29

[F21] *Digital [F: Steps*
Taken from [13] Chapter 9: Security. p. 74. 30

[F22] *MD5 Collision demo visualized*
Calculated on md5hashgenerator.com in June 2021. 31

[F23] *Manipulating the low-order bits of an image file and hiding information inside the pixels*
Taken from [13] Chapter 9: Security. p. 76. 32

[F24] *Cryptographic key visualized in binary format*
Taken from encrypted-tbn0.gstatic.com/images?q=tbn:ANd9GcTUMUNwXMGgHvbWpAdE42rVK7vV0BbrSFxWFNjhWirYeUXbOaT5&s in June 2021. 33

[F25] *The Steps of the Diffie-Hellman Protocol*
Taken from [13] Chapter 9: Security. p. 81. 34

[F26] *Man-in-the-middle attack visualized*
Taken from [13] Chapter 9: Security. p. 83. 35

[F27] *Web of Trust visualized*
Taken from [13] Chapter 9: Security. p. 85. 35

[F28] *Hierarchy of Certification Authorities*
Taken from [13] Chapter 9: Security. p. 84. 36

[F29] *FIDO U2F authentication workflow*
Taken from upload.wikimedia.org/wikipedia/commons/thumb/4/47/Auth_flow4.svg.png in June 2021. 39

[F30] *FIDO UAF registration message flow*
Taken from
fidoalliance.org/specs/fido-uaf-v1.1-id-20170202/fido-uaf-overview-v1.1-id-20170202.html in June 2021. 40

[F31] *CTAP authentication overview*
Taken from
doubleoctopus.com/wp-content/uploads/2019/09/CTAP-authentication.png in June 2021. 41

[F32] *Raven One SimuLink overview* 42

[F33] *Raven One SimuLink RSA uml type class diagram* 44

[F34] *Raven One SimuLink MD5 uml type class diagram* 45

[F35] *Raven One SimuLink main gui uml type activity diagram* 46

[F36] *Raven One SimuLink RSA uml type activity diagram* 47

[F37] *Jamestown Online Kundenportal overview* 49

[F38] *FIDO passwordless collage* 50
Taken from fidoalliance.org/fido2/ in June 2021.

[F39] *Jamestown Online-Beitritt Zeichnungsschein overview* 51

[F40] *Cryptology definition* 52

10 List of Tables

[T1] Passwords overview advantages and disadvantages of various techniques 20

11 References

[1] Grassi, Paul A.; Garcia, Michael E.; Fenton, James L.: **Digital Identity Guidelines**. NIST Special Publication 800-63-3, February 2020.

[2] Grassi, Paul A.; Fenton, James L.: **Digital Identity Guidelines** *Enrollment and Identity Proofing*. NIST Special Publication 800-63A, February 2020.

[3] Grassi, Paul A.; Fenton, James L.; Newton, Elaine M.; Perlner, Ray A.; Regenscheid, Andrew R.; Burr, William E.; Richer, Justin P.: **Digital Identity Guidelines** *Authentication and Lifecycle Management*. NIST Special Publication 800-63B, February 2020.

[4] ISO/IEC 27000:2018(E), **Information technology - Security techniques - Information security management systems**. Fifth edition, February 2018.

[5] Microsoft Corporation: **Password-less protection**. Accessed: 2021-04 to 2021-06 at microsoft.com/security/business/identity-access-management/passwordless-authentication.

[6] Tanwar, Rohit; Chowhan, Rahul Singh: **Password-Less Authentication:** *Methods for User Verification and Identification to Login Securely Over Remote Sites*. ResearchGate, January 2019.

[7] Teh, Pin Shen; Zhang, Ning; Tan, Syh-Yuan; Shi, Qi; Khoh, Wee How; Nawaz, Raheel: **Strengthen user authentication on mobile devices by using user's touch dynamics pattern**. SpringerLink, December 2019.

[8] **Passwordless Authentication** *The next breakthrough in secure digital transformation*. World Economic Forum in collaboration with the FIDO Alliance, January 2020.

[9] FIDO Alliance: **FIDO2**. Accessed: 2021-04 to 2021-06 at fidoalliance.org/fido2.

[10] Jamestown: **Jamestown Online**. Accessed: 2021-04 to 2021-06 at online.jamestown.de.

[11] Laumann, Markus; Kabatas, Serkan: **Nevis Authentication Cloud**. Presentation at Jamestown, March 2021.

[12] Tsolkas, Alexander; Schmidt, Klaus: **Rollen und Berechtigungskonzepte** *Ansätze für das Identity- und Access Management im Unternehmen*. Vieweg+Teubner edition <kes>, 2010.

[13] Vogt, C.: **Operating Systems and Distributed Systems lecture**. TH Köln, 2019.

[14] FIDO Alliance: **Authenticate 2020 Conference**. Virtual presentation, November 2020.

[15] Wikipedia: **Ali Baba and the Forty Thieves**. Accessed: 2021-04 to 2021-06 at wikipedia.org/wiki/Ali_Baba_and_the_Forty_Thieves.

[16] Jain, A. K.; Bolle, R.; Pankanti, S.: **Biometrics** *Personal Identification in Network Society*. Kluwer Academic Publications, 1999.

[17] Wikipedia: **Cryptography**. Accessed: 2021-04 to 2021-06 at wikipedia.org/wiki/Cryptography.

[18] Wikipedia: **One-time pad**. Accessed: 2021-04 to 2021-06 at wikipedia.org/wiki/One-time_pad.

[19] Shannon, Claude E.: **Communication Theory of Secrecy Systems**. Bell Technical Journal, 1949.

[20] Wikipedia: **Information-theoretic security**. Accessed: 2021-04 to 2021-06 at wikipedia.org/wiki/Information-theoretic_security.

[21] Bernstein, Daniel J.: **Introduction to post-quantum cryptography**. Department of Computer Science, University of Illinois at Chicago, 2009.

[22] Kerckhoffs, A.: **Kerckhoffs' Principle**. SpringerLink, 1883.

[23] Wikipedia: **Advanced Encryption Standard**. Accessed: 2021-04 to 2021-06 at wikipedia.org/wiki/Advanced_Encryption_Standard.

[24] Selinger, Peter: **MD5 Collision Demo**. Accessed: 2021-04 to 2021-06 at mscs.dal.ca/~selinger/md5collision.

12 Statutory Declaration

I herewith formally declare that I have produced the submitted thesis independently. I did not use any outside support except for the quoted literature and other sources mentioned in the paper.

I clearly marked and separately recorded all of the literature and all of the other sources which I employed when producing this academic work, either literally or in content.

I am aware that the violation of this regulation will lead to failure of the thesis.

Mert Torun	
Student's name	Student's signature
Matriculation number	*Cologne, 30th June 2021*

Technology
Arts Sciences
TH Köln

Jamestown

13 Blocking Notice

This bachelor thesis with the title ***Password- / Keyless authentication*** is based on and may contain internal, confidential data and information of *Jamestown US-Immobilien GmbH*.

The access to this thesis is restricted until ***30th June 2024***.

This work may only be available to the first and second reviewers and authorized members of the board of examiners of the TH Köln until the access restrictions are lifted. Any publication and duplication of this bachelor thesis — even in part — are prohibited.

An inspection of this work by third parties requires the expressed admission of the author and the company.

Mert Torun

Student's name

Student's signature

Matriculation number

Cologne, 30th June 2021

TH Köln
Gustav-Heinemann-Ufer 54
50968 Köln
www.th-koeln.de

Technology
Arts Sciences
TH Köln